ADVANCED MACHINE LEARNING · RESEARCH PAPER PROJECT 23

# Induction of Decision Trees

ID3 from scratch on 7 real datasets: information gain, depth control and noise

J. R. Quinlan (1986), Machine Learning 1(1): 81–106 · doi:10.1007/BF00116251

GROUP

Vinayak Singh (22MA25005) · Debjit Mondal (22MA25009) · Keya Battu (22MA25006) · Gurkirat Singh (22MA25015)
Nikhil Nagar (22MA25001) · Mayank Fenin (22MA25002) · Atharv Sable (21MA20001)

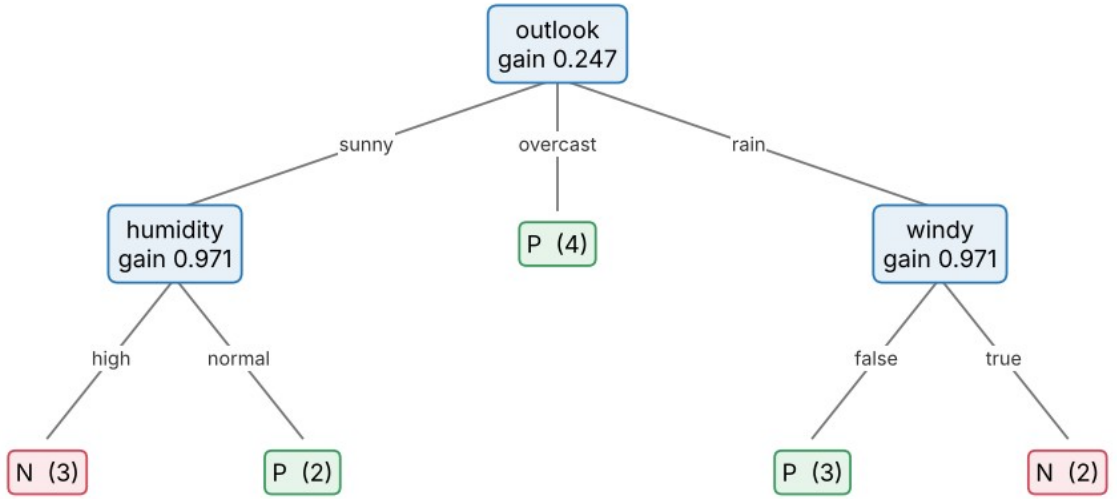
Code and data: github.com/Vinayak069/id3-decision-trees

MA30235 / MA60215 · Department of Mathematics, IIT Kharagpur · November 2026

# Learn a small tree that classifies objects from their attributes

| outlook | temperature | humidity | windy | class |
|---|---|---|---|---|
| sunny | hot | high | false | N |
| sunny | hot | high | true | N |
| overcast | hot | high | false | P |
| rain | mild | high | false | P |
| rain | cool | normal | false | P |
| rain | cool | normal | true | N |
| overcast | cool | normal | true | P |
| sunny | mild | high | false | N |
| sunny | cool | normal | false | P |
| rain | mild | normal | false | P |
| sunny | mild | normal | true | P |
| overcast | mild | high | true | P |
| overcast | hot | normal | false | P |
| rain | mild | high | true | N |

Table 1 of the paper: 14 Saturday mornings, P = suitable, N = not.




- **Induction task:** given objects described by categorical attributes and a class, build a rule that classifies unseen objects.
- **Top-down divide and conquer:** pick a test, split the objects, recurse until every subset is one class.
- **Occam's razor:** among trees that fit, prefer the simplest. ID3 gets there greedily, without searching all trees.

# ID3 splits on the attribute that carries most information about the class

$$I(C) = -\sum_k \frac{c_k}{|C|} \log_2 \frac{c_k}{|C|}$$

$$E(A) = \sum_{i=1}^{v} \frac{|C_i|}{|C|}\ I(C_i)$$

$$\text{gain}(A) = I(C) - E(A)\ =\ \text{MI}(A;\ \text{class})$$

$$IV(A) = -\sum_i \frac{|C_i|}{|C|} \log_2 \frac{|C_i|}{|C|}, \quad \text{ratio} = \frac{\text{gain}(A)}{IV(A)}$$

Worked root split: E(outlook) = 5/14 · 0.971 + 4/14 · 0 + 5/14 · 0.971 = 0.694
gain(outlook) = 0.940 − 0.694 = 0.246 bits (0.2467 unrounded)

| Attribute | gain (ours) | gain (paper) | IV | gain ratio | $\chi^2$ p-value |
|---|---|---|---|---|---|
| outlook | 0.247 | 0.246 | 1.577 | 0.156 | 0.170 |
| temperature | 0.029 | 0.029 | 1.557 | 0.019 | 0.752 |
| humidity | 0.152 | 0.151 | 1.000 | 0.152 | 0.094 |
| windy | 0.048 | 0.048 | 0.985 | 0.049 | 0.334 |

- **I(9, 5) = 0.940 bits;** outlook wins with 0.247 bits, so it becomes the root. Our values match the paper to its rounding.
- **Gain is mutual information** (paper footnote 3): ID3 asks which test most reduces uncertainty about the class.
- **Bias:** gain rewards attributes with many values. The gain ratio divides by IV(A) (Section 7): outlook 0.156 vs humidity 0.152.

METHOD

# We implemented every mechanism in the paper, then tested it

### Section 4 · Tree + window

- Recursive split on max gain; each attribute used once per path
- Empty branch → majority class of parent
- Windowing: grow on a sample, add misclassified objects, repeat

### Section 5 · Noise

- Impure leaves are labelled with the majority class
- $\chi^2$ test of independence: no split unless irrelevance is rejected ($\alpha$ = 1 % in the paper; we sweep $\alpha$)

### Section 6 · Unknown values

- Gain: spread unknowns over values $\propto |C_i|$, then discard them
- Classify: a token goes down every branch with weight $\text{ratio}_i$; class votes are summed

$$\chi^2 = \sum_i \sum_k \frac{(c_{ik} - c'_{ik})^2}{c'_{ik}}, \qquad c'_{ik} = c_k \frac{|C_i|}{|C|}$$

$$\text{split only if } \; P\left(\chi^2_{(v-1)(K-1)} > \chi^2\right) < 0.01$$

Pure Python / NumPy, about 500 lines (id3.py). sklearn is used only for the CART baseline.

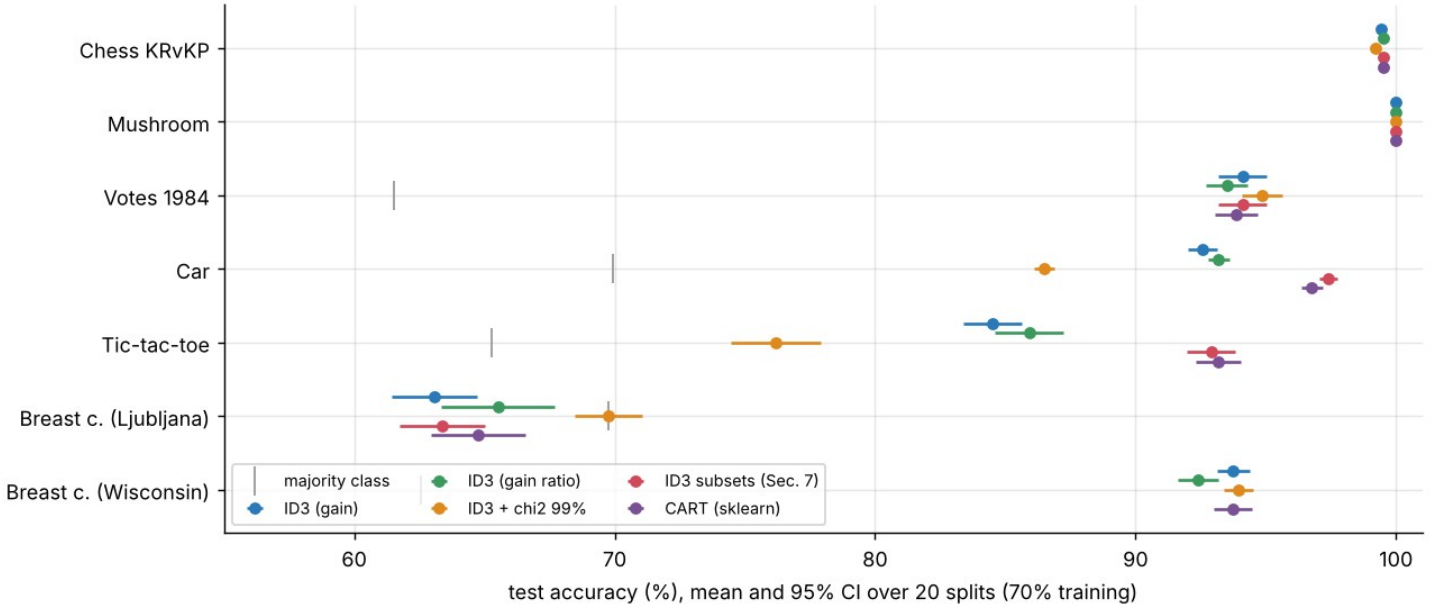
### Section 7 · Extensions

- Gain ratio among attributes with ≥ average gain
- Binary subset tests A ∈ S (ASSISTANT)
- Continuous attributes: threshold at midpoints

### Our additions

- max_depth for the depth study; K ≥ 2 classes
- 15 unit tests of the paper's claims
- Corrected paired t-test (Nadeau–Bengio)

# On 7 real datasets ID3 with subset tests matches CART

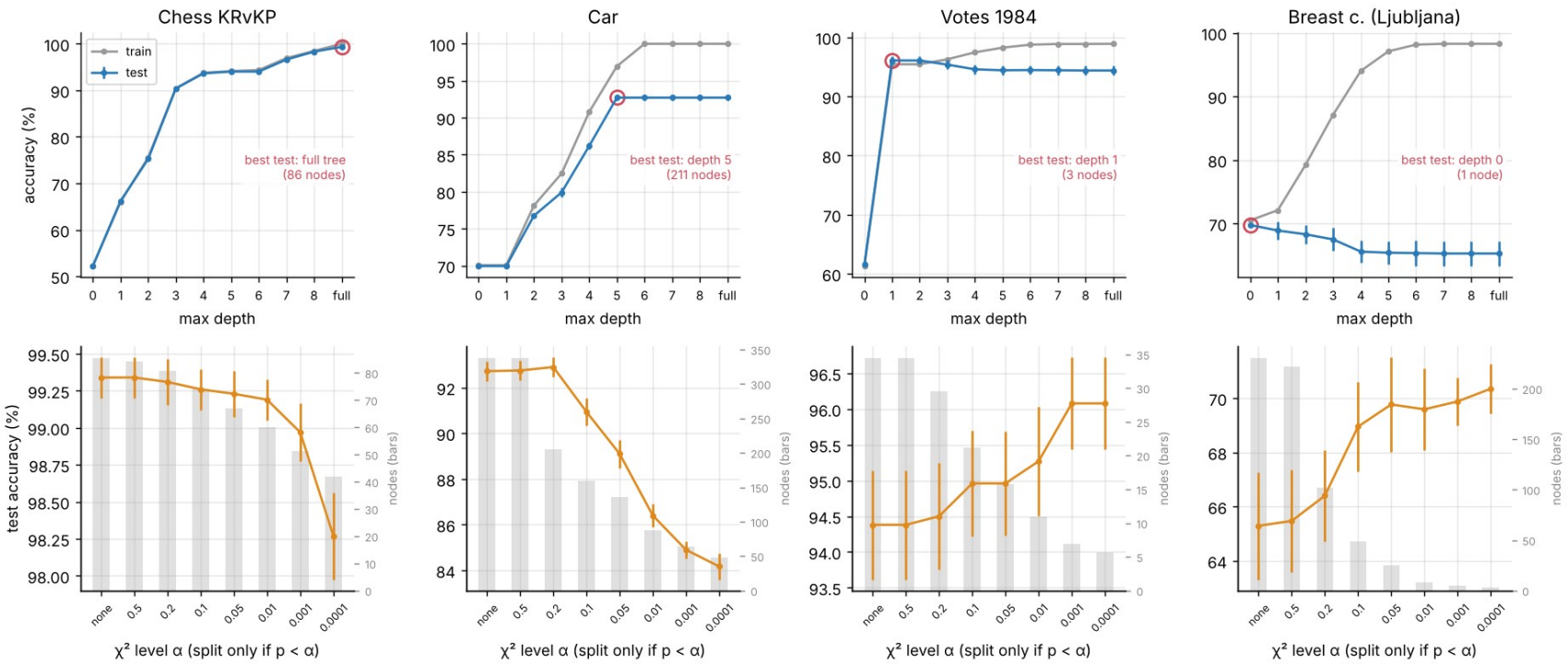



| Dataset | Objects | Attrs | Classes | '?' values |
|---|---|---|---|---|
| Chess KRvKP | 3 196 | 36 | 2 | 0 |
| Mushroom | 8 124 | 22 | 2 | 2480 |
| Votes 1984 | 435 | 16 | 2 | 392 |
| Car Evaluation | 1 728 | 6 | 4 | 0 |
| Tic-tac-toe | 958 | 9 | 2 | 0 |
| Breast c. Ljubljana | 286 | 9 | 2 | 9 |
| Breast c. Wisconsin | 569 | 30 | 2 | 0 |
| Sim: known tree | 2 000 | 10 | 2 | 0 |
| Sim + id_code | 600 | 11 | 2 | 0 |

UCI data (GitHub mirrors) + scikit-learn; simulated sets only for sanity checks.

- **Fragmentation is ID3's weak spot:** on Car and Tic-tac-toe, multi-way ID3 trails CART by 4–9 points. The paper's own binary subset tests close the gap (+4.8 and +8.4 points, 20/20 and 20/20 wins, $p < 0.001$).
- **Clean concepts (chess, mushroom):** all variants reach 99.4–100 %; differences are not significant.
- **Noisy medical data (Ljubljana):** the $\chi^2$ test gains 6.7 points over the full tree ($p = 0.04$) with 8 nodes instead of 224, but only reaches the majority-class rate: the attributes carry little signal.

# Depth control: shallow trees win on noisy data, full trees on clean data

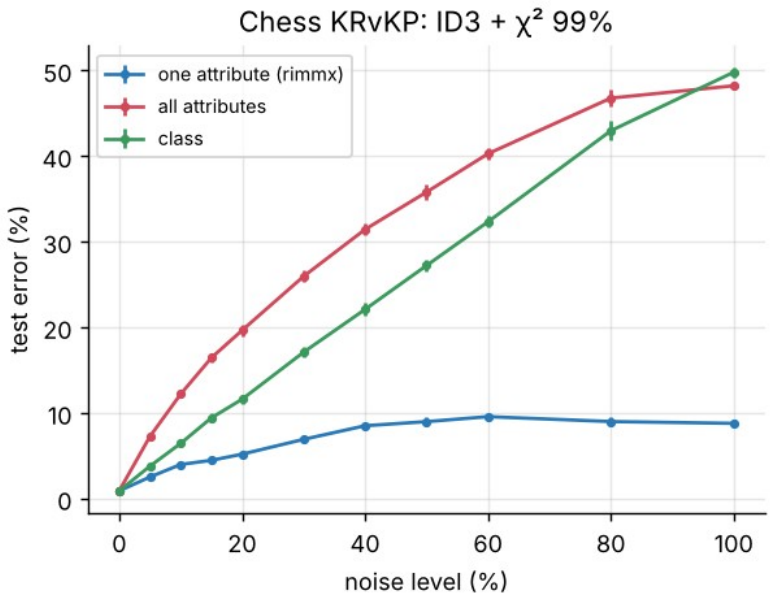


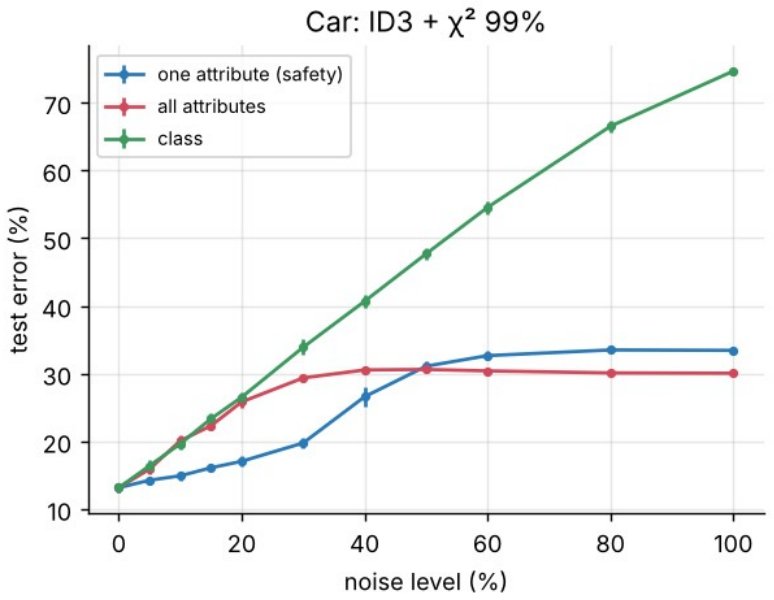

## How to read it

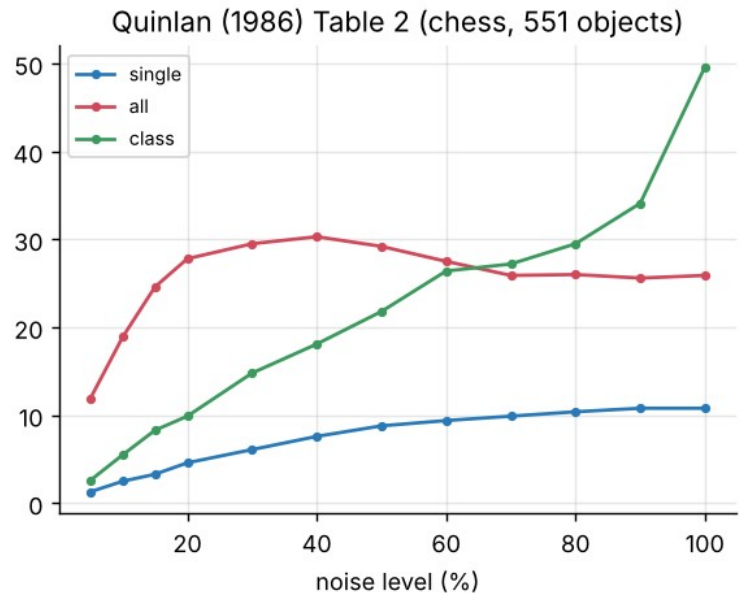
- Top: max_depth 0 … 8 and the full tree (train grey, test blue)
- Bottom: $\chi^2$ level α from 'none' to 0.0001; bars = tree size
- 20 stratified 70/30 splits, 95 % CI
- Ljubljana: best at depth 0, the majority class (69.8 %); the full tree has 231 nodes and 65.3 %

- **Clean concepts:** chess and car need deep trees (chess depth 3: 90.4 %, full: 99.3 %); a strict $\chi^2$ test costs car 8.6 points.
- **Noisy concepts:** votes is best at depth 1, one question about physician-fee-freeze (96.1 % vs 94.4 %, 3 nodes vs 34). Training accuracy keeps rising while test accuracy falls: overfitting.

# Noise on real data reproduces the shapes of the paper's Table 2



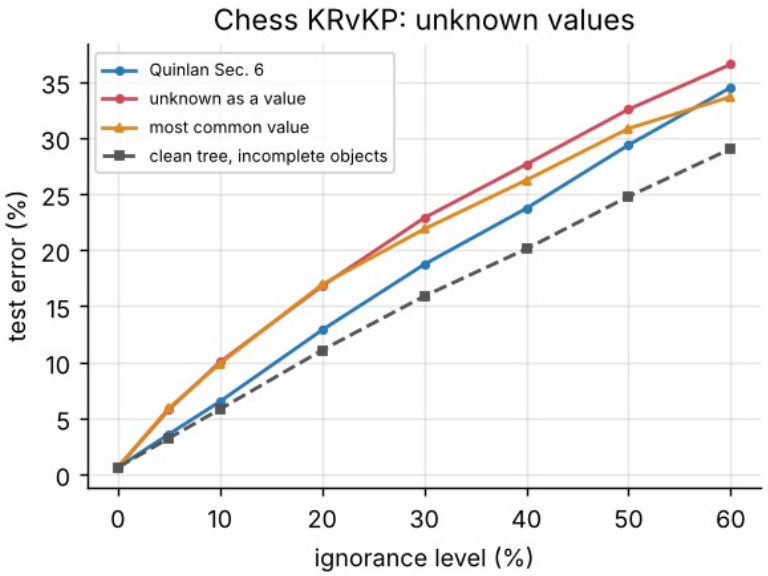



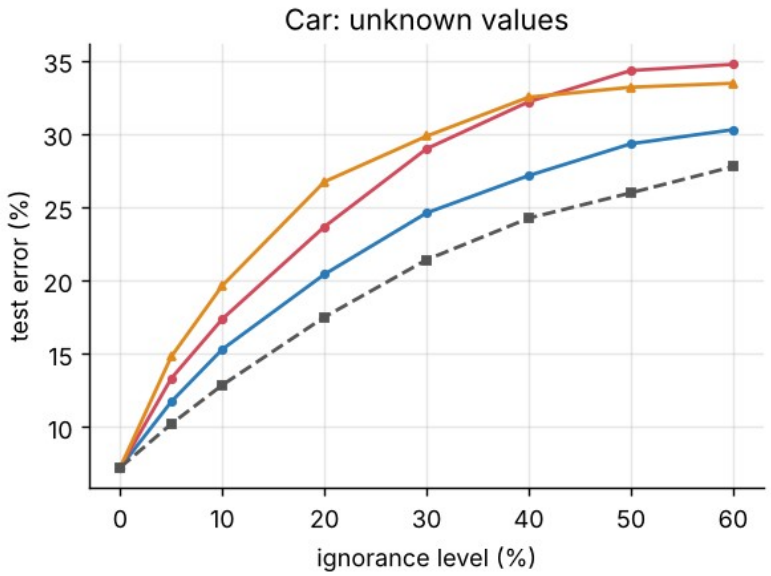

- **Class noise** grows almost linearly to random guessing: chess 49.8 % at 100 %, the paper 49.6 %.
- **Noise in all attributes** on car levels off at the majority-class error (30.1 %) once $\chi^2$ finds nothing relevant, as in the paper.
- **One attribute** (the root attribute) saturates: chess rises only to 8.8 %, the paper 10.8 %.

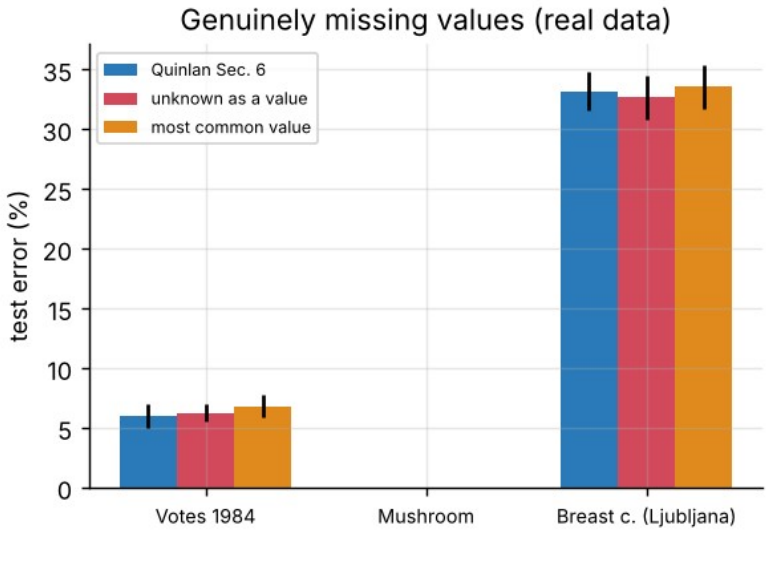
### Why $\chi^2$ matters (Section 5)

- Probabilistic leaf errs $2pn/(p+n)^2$; majority leaf $n/(p+n)$, smaller when $p > n$
- Ablation on chess with 20 % class noise: without $\chi^2$ 1160 nodes and 20.5 % error; with $\chi^2$ 53 nodes and 11.6 %

# Unknown values, gain bias and what we learned




- **Unknowns:** at 30 % ignorance on car, Quinlan's rule errs 24.6 % vs 29.0 % ('?' as a value) and 29.9 % (mode). On genuinely missing data the three tie.
- **Gain bias:** an irrelevant 150-valued id_code enters the gain tree (171 nodes, 95.5 %); the gain ratio never uses it (13 nodes, 100 %).
- **Windowing:** 304 of 2 237 chess objects gave a tree with zero training errors.

## Conclusions

- Code reproduces the paper's numbers and recovers a known tree
- Depth limits / $\chi^2$ help noisy data, hurt clean data
- Subset tests fix multi-way fragmentation
- github.com/Vinayak069/id3-decision-trees

# Appendix: code, data and outputs

This appendix follows the 8 presentation slides. It contains (A) how to reproduce the results, (B) data details, (C) all output tables and figures and (D) the full code listing, including the unit tests. Every number comes from `python experiments.py` with fixed seeds (20 stratified 70/30 splits per setting). Code and data: `https://github.com/Vinayak069/id3-decision-trees`.

Group: Vinayak Singh (22MA25005), Debjit Mondal (22MA25009), Keya Battu (22MA25006), Gurkirat Singh (22MA25015), Nikhil Nagar (22MA25001), Mayank Fenin (22MA25002), Atharv Sable (21MA20001).

## A. How to run and reproduce

Colab (for the live demo): either upload the folder `id3-project` to My Drive, or run `!git clone https://github.com/Vinayak069/id3-decision-trees` and `%cd id3-decision-trees`. Open `ID3_Project.ipynb` and run all cells. Set `QUICK = True` to use 5 repetitions instead of 20 (a few minutes).

Laptop: `pip install -r requirements.txt`, then `python -m pytest -q tests` (15 tests, about 1 s), `python simulate.py` (regenerates the simulated CSVs) and `python experiments.py`, which writes results/*.csv and figures/*.png in about 8 minutes on 4 cores (`--quick` for 5 repetitions). scikit-learn is used only for the CART baseline and to load the Wisconsin data.

| File | Purpose |
|---|---|
| id3.py | ID3 from scratch: entropy, gain, gain ratio, $\chi^2$ test, unknown values, windowing, binary subset tests, continuous thresholds, noise injection |
| simulate.py | Simulated data from a known decision tree (seeds 2026 and 7), used only as a sanity check |
| experiments.py | All experiments; writes every table and figure |
| tests/test_id3.py | 15 unit tests, each checking a number or claim from the paper |
| ID3_Project.ipynb | Demo notebook: theory, experiments, figures and discussion |
| data/real/*.csv | The paper's Table 1 and 7 real UCI datasets |
| data/simulated/*.csv | sim_tree_2000.csv, sim_with_id_600.csv |
| results/, figures/ | Outputs reproduced below |

## B. Data details

| dataset | file | type | objects | attributes | values per attribute | classes | unknown values |
|---|---|---|---|---|---|---|---|
| Saturday mornings (paper Table 1) | real/weather_quinlan1986.csv | real | 14 | 4 | 2-3 | P 9, N 5 | 0 |
| Chess endgame KR vs KP (UCI) | real/chess_kr_vs_kp.csv | real | 3196 | 36 | 2-3 | won 1669, nowin 1527 | 0 |
| Mushroom (UCI) | real/mushrooms.csv | real | 8124 | 22 | 1-12 | e 4208, p 3916 | 2480 |
| Congressional votes 1984 (UCI) | real/house_votes_84.csv | real | 435 | 16 | 3-3 | democrat 267, republican 168 | 392 |
| Car Evaluation (UCI) | real/car_evaluation.csv | real | 1728 | 6 | 3-4 | unacc 1210, acc 384, good 69, vgood 65 | 0 |
| Tic-tac-toe endgame (UCI) | real/tic_tac_toe.csv | real | 958 | 9 | 3-3 | positive 626, negative 332 | 0 |
| Breast cancer, Ljubljana (UCI) | real/breast_cancer_ljubljana.csv | real | 286 | 9 | 2-11 | no-recurrence-events 201, recurrence-events 85 | 9 |
| Breast cancer, Wisconsin (UCI) | real/breast_cancer_wisconsin.csv | real | 569 | 30 | continuous | benign 357, malignant 212 | 0 |
| Simulated: known depth-3 tree | simulated/sim_tree_2000.csv | simulated | 2000 | 10 | 3-3 | P 1178, N 822 | 0 |
| Simulated + 150-valued id_code | simulated/sim_with_id_600.csv | simulated | 600 | 11 | 3-150 | P 339, N 261 | 0 |

| Dataset | Original source | Copy used (UCI site unreachable) |
|---|---|---|
| Chess KR vs KP | UCI, Shapiro 1987, doi:10.24432/C5DK5C | github: mikeizbicki/datasets |
| Mushroom | UCI, Schlimmer 1981, doi:10.24432/C5959T | github: stedy/Machine-Learning-with-R-datasets |
| Congressional votes 1984 | UCI, Schlimmer 1987, doi:10.24432/C5C01P | github: mhahsler/arules |
| Car Evaluation | UCI, Bohanec & Rajkovic 1988, doi:10.24432/C5JP48 | github: scikit-learn-contrib/category_encoders |
| Tic-tac-toe endgame | UCI, Aha 1991, doi:10.24432/C5688J | github: LucasBoTang/Optimal_Classification_Trees |

| Dataset | Original source | Copy used (UCI site unreachable) |
|---|---|---|
| Breast cancer (Ljubljana) | UCI, Zwitter & Soklic 1988, doi:10.24432/C51P4M | github: LucasBoTang/Optimal_Classification_Trees |
| Breast cancer (Wisconsin) | UCI, Wolberg et al. 1995, doi:10.24432/C5DW2B | scikit-learn load_breast_cancer |

Saturday mornings: typed from Table 1 of the paper. Chess KR vs KP is the chess endgame domain of the paper itself (Section 4): 36 board features, won or not won for white. Mushroom: `stalk_root` is unknown for 2 480 records. Votes 1984: 16 key votes of the US House; '?' (392 values) means the member did not vote. Car Evaluation: 4 classes, a hierarchical concept that multi-way splits fragment. Tic-tac-toe: all legal end boards; the class is a disjunction of 8 lines. Ljubljana breast cancer: recurrence after surgery; 9 values unknown; known to be very noisy. Wisconsin breast cancer: 30 continuous features, handled with Section 7 thresholds. In all files '?' marks an unknown value.

Simulated data (sanity checks only): ten independent attributes A1–A10, uniform on {a, b, c}. The class is given by the tree below; A5–A10 are irrelevant. P(class = P) = 16/27 = 0.593. `sim_with_id_600.csv` adds `id_code`, an irrelevant attribute with 150 random values, to test the Section 7 gain bias.

```
A1 = a:
|   A2 = a: P
|   A2 = b: N
|   A2 = c: P
A1 = b: P
A1 = c:
|   A3 = a: N
|   A3 = b: N
|   A3 = c:
|   |   A4 = a: P
|   |   A4 = b: N
|   |   A4 = c: N
```

Protocol: stratified random splits, 70 % train and 30 % test, 20 repetitions with fixed seeds, mean ± standard error. Paired comparisons use the same splits for every method and the Nadeau–Bengio corrected resampled t-test. Noise: with probability n a value is replaced by a value drawn uniformly from the attribute's (or class's) domain (paper footnote 5); training and test parts are corrupted independently.

Departures from the paper (labelled): (1) entropy, gain and the $\chi^2$ statistic are written for $K \geq 2$ classes and reduce to the paper's formulas when $K = 2$; (2) `max_depth` and `min_samples_split` are our additions for the depth-control study; (3) the paper's own chess files (e.g. 551 objects) are not public, so we use the public KR vs KP endgame and six other UCI datasets; (4) for noise and unknown values we corrupt the training and test parts of a 70/30 split independently, whereas the paper corrupted the whole set twice; (5) CART (scikit-learn) and the majority class are added as baselines, and methods are compared with the Nadeau–Bengio corrected t-test.

# C. Outputs

## E1. The paper's worked example (Table 1)

| attribute | E(A) bits | gain bits | paper gain | IV bits | paper IV | gain ratio | paper gain ratio | chi2 | p-value |
|---|---|---|---|---|---|---|---|---|---|
| outlook | 0.6935 | 0.2467 | 0.246 | 1.5774 | 1.578 | 0.1564 | 0.156 | 3.5467 | 0.1698 |
| temperature | 0.9111 | 0.0292 | 0.029 | 1.5567 | — | 0.0188 | — | 0.5704 | 0.7519 |
| humidity | 0.7885 | 0.1518 | 0.151 | 1.0 | 1.0 | 0.1518 | 0.151 | 2.8 | 0.0943 |
| windy | 0.8922 | 0.0481 | 0.048 | 0.9852 | — | 0.0488 | — | 0.9333 | 0.334 |

```
I(9,5) = 0.9403 bits (paper 0.940)

ID3 tree (gain):
outlook = overcast: P  (n=4)
outlook = rain:
|   windy = false: P  (n=3)
|   windy = true: N  (n=2)
outlook = sunny:
|   humidity = high: N  (n=3)
|   humidity = normal: P  (n=2)

Training accuracy 1.000; nodes 8, leaves 5

With the 99% chi-square test the tree is a single leaf:
-> P  N: 5, P: 9
(14 objects are too few to reject independence)
```

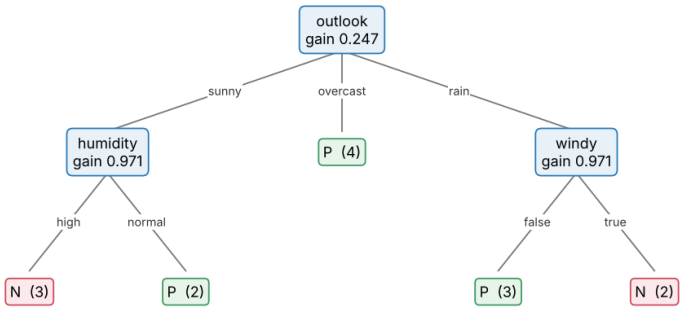



Figure A1. ID3 tree learned from Table 1: identical to the paper's Figure 2.

## Sanity check on simulated data

```
ID3 on 2000 simulated objects recovers the generating tree exactly: True

A1 = a:
|   A2 = a: P  (n=226)
|   A2 = b: N  (n=215)
|   A2 = c: P  (n=202)
A1 = b: P  (n=677)
A1 = c:
|   A3 = a: N  (n=241)
|   A3 = b: N  (n=224)
|   A3 = c:
|   |   A4 = a: P  (n=73)
|   |   A4 = b: N  (n=69)
|   |   A4 = c: N  (n=73)
```

## E2. Benchmark on 7 real datasets (20 splits, 70 % training)

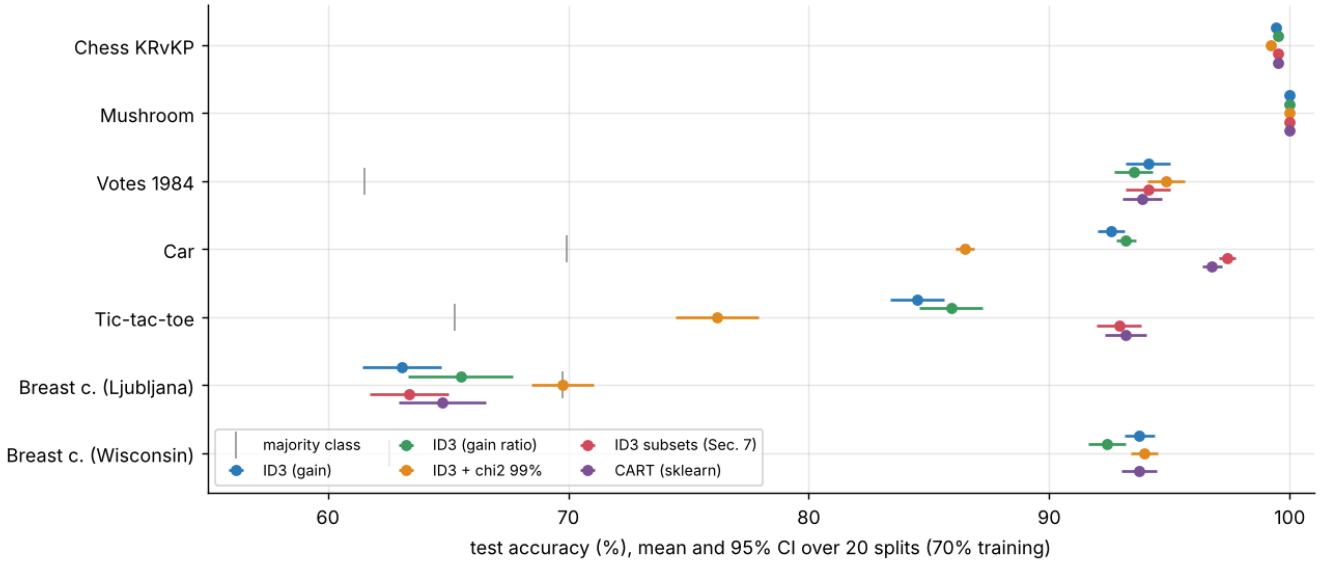



Figure A2. Test accuracy, mean and 95 % CI.

| dataset | model | accuracy % | nodes |
|---|---|---|---|
| chess | ID3 (gain) | 99.42 ± 0.07 | 87.8 |
| chess | ID3 (gain ratio) | 99.53 ± 0.06 | 87.9 |
| chess | ID3 + chi2 99% | 99.20 ± 0.07 | 60.6 |
| chess | ID3 subsets (Sec. 7) | 99.49 ± 0.06 | 82.1 |
| chess | ID3 subsets + chi2 99% | 99.26 ± 0.06 | 55.8 |
| chess | CART (sklearn) | 99.49 ± 0.06 | 82.0 |
| chess | Majority class | 52.24 ± 0.00 | 1.0 |
| mushroom | ID3 (gain) | 100.00 ± 0.00 | 38.0 |
| mushroom | ID3 (gain ratio) | 100.00 ± 0.00 | 29.2 |
| mushroom | ID3 + chi2 99% | 100.00 ± 0.00 | 38.0 |
| mushroom | ID3 subsets (Sec. 7) | 100.00 ± 0.00 | 15.9 |
| mushroom | ID3 subsets + chi2 99% | 100.00 ± 0.00 | 15.9 |
| mushroom | CART (sklearn) | 99.99 ± 0.01 | 22.8 |
| mushroom | Majority class | 51.78 ± 0.00 | 1.0 |
| votes | ID3 (gain) | 94.12 ± 0.47 | 34.2 |
| votes | ID3 (gain ratio) | 93.50 ± 0.40 | 36.1 |
| votes | ID3 + chi2 99% | 94.85 ± 0.40 | 9.7 |
| votes | ID3 subsets (Sec. 7) | 94.12 ± 0.47 | 34.2 |
| votes | ID3 subsets + chi2 99% | 94.85 ± 0.40 | 9.7 |
| votes | CART (sklearn) | 93.85 ± 0.42 | 37.4 |
| votes | Majority class | 61.54 ± 0.00 | 1.0 |
| car | ID3 (gain) | 92.57 ± 0.28 | 334.45 |
| car | ID3 (gain ratio) | 93.18 ± 0.21 | 332.95 |
| car | ID3 + chi2 99% | 86.48 ± 0.20 | 88.65 |
| car | ID3 subsets (Sec. 7) | 97.41 ± 0.18 | 136.8 |
| car | ID3 subsets + chi2 99% | 95.21 ± 0.19 | 65.2 |
| car | CART (sklearn) | 96.77 ± 0.21 | 155.0 |
| car | Majority class | 69.94 ± 0.00 | 1.0 |
| tictactoe | ID3 (gain) | 84.50 ± 0.57 | 270.1 |
| tictactoe | ID3 (gain ratio) | 85.92 ± 0.67 | 263.65 |
| tictactoe | ID3 + chi2 99% | 76.18 ± 0.88 | 53.5 |
| tictactoe | ID3 subsets (Sec. 7) | 92.90 ± 0.47 | 125.2 |
| tictactoe | ID3 subsets + chi2 99% | 89.15 ± 1.16 | 43.9 |
| tictactoe | CART (sklearn) | 93.18 ± 0.45 | 124.7 |
| tictactoe | Majority class | 65.28 ± 0.00 | 1.0 |
| bcl | ID3 (gain) | 63.08 ± 0.84 | 224.15 |

| dataset | model | accuracy % | nodes |
|---|---|---|---|
| bcl | ID3 (gain ratio) | 65.52 ± 1.11 | 214.15 |
| bcl | ID3 + chi2 99% | 69.77 ± 0.65 | 8.3 |
| bcl | ID3 subsets (Sec. 7) | 63.37 ± 0.83 | 119.4 |
| bcl | ID3 subsets + chi2 99% | 68.90 ± 0.94 | 9.2 |
| bcl | CART (sklearn) | 64.77 ± 0.93 | 128.7 |
| bcl | Majority class | 69.77 ± 0.00 | 1.0 |
| wdbc | ID3 (gain) | 93.74 ± 0.32 | 27.9 |
| wdbc | ID3 (gain ratio) | 92.40 ± 0.40 | 35.8 |
| wdbc | ID3 + chi2 99% | 93.95 ± 0.29 | 22.3 |
| wdbc | CART (sklearn) | 93.74 ± 0.37 | 28.0 |
| wdbc | Majority class | 62.57 ± 0.00 | 1.0 |

## Paired comparisons (difference in accuracy points; corrected resampled t-test)

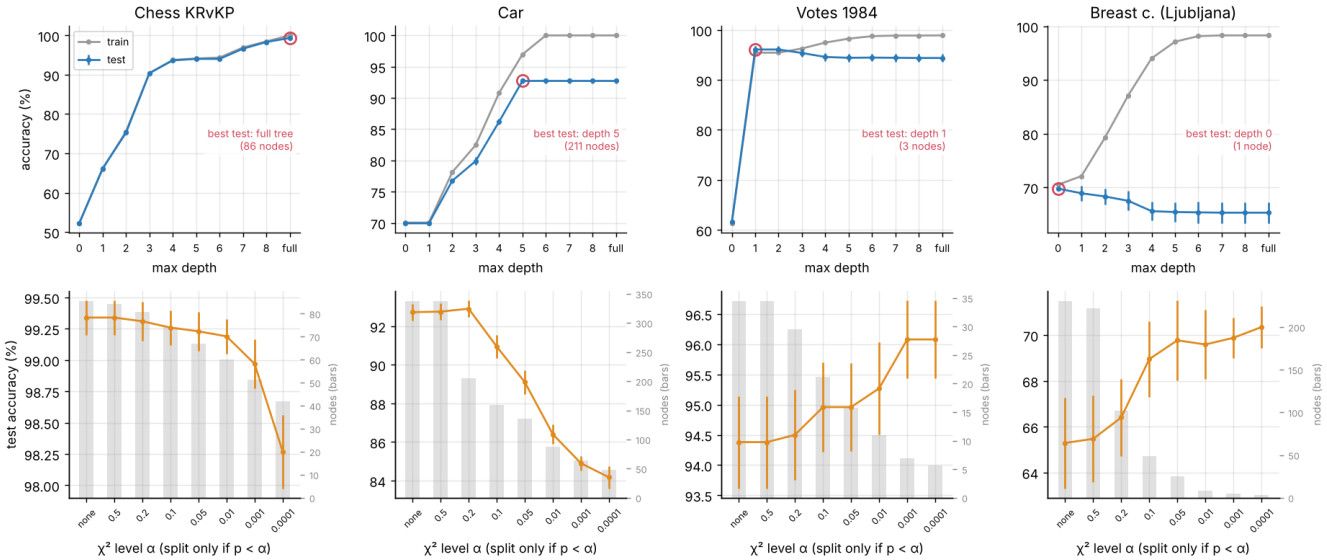
| dataset | comparison | mean_diff_pts | wins | ties | losses | t | p_value |
|---|---|---|---|---|---|---|---|
| chess | ID3 subsets (Sec. 7) - ID3 (gain) | 0.073 | 6 | 13 | 1 | 0.733 | 0.4725 |
| chess | ID3 subsets (Sec. 7) - CART (sklearn) | 0.0 | 1 | 17 | 2 | 0.0 | 1.0 |
| chess | ID3 (gain) - CART (sklearn) | -0.073 | 2 | 11 | 7 | -0.7135 | 0.4842 |
| chess | ID3 + chi2 99% - ID3 (gain) | -0.219 | 1 | 2 | 17 | -1.9991 | 0.0601 |
| chess | ID3 (gain ratio) - ID3 (gain) | 0.1095 | 9 | 7 | 4 | 0.5182 | 0.6103 |
| mushroom | ID3 subsets (Sec. 7) - ID3 (gain) | 0.0 | 0 | 20 | 0 | 0.0 | 1.0 |
| mushroom | ID3 subsets (Sec. 7) - CART (sklearn) | 0.0082 | 2 | 18 | 0 | 0.4697 | 0.6439 |
| mushroom | ID3 (gain) - CART (sklearn) | 0.0082 | 2 | 18 | 0 | 0.4697 | 0.6439 |
| mushroom | ID3 + chi2 99% - ID3 (gain) | 0.0 | 0 | 20 | 0 | 0.0 | 1.0 |
| mushroom | ID3 (gain ratio) - ID3 (gain) | 0.0 | 0 | 20 | 0 | 0.0 | 1.0 |
| votes | ID3 subsets (Sec. 7) - ID3 (gain) | 0.0 | 0 | 20 | 0 | 0.0 | 1.0 |
| votes | ID3 subsets (Sec. 7) - CART (sklearn) | 0.2692 | 9 | 4 | 7 | 0.2205 | 0.8279 |
| votes | ID3 (gain) - CART (sklearn) | 0.2692 | 9 | 4 | 7 | 0.2205 | 0.8279 |
| votes | ID3 + chi2 99% - ID3 (gain) | 0.7308 | 10 | 2 | 8 | 0.5244 | 0.6061 |
| votes | ID3 (gain ratio) - ID3 (gain) | -0.6154 | 2 | 10 | 8 | -0.8517 | 0.405 |
| car | ID3 subsets (Sec. 7) - ID3 (gain) | 4.8362 | 20 | 0 | 0 | 4.5739 | 0.0002 |
| car | ID3 subsets (Sec. 7) - CART (sklearn) | 0.6358 | 16 | 0 | 4 | 1.0887 | 0.2899 |
| car | ID3 (gain) - CART (sklearn) | -4.2004 | 0 | 0 | 20 | -3.6959 | 0.0015 |
| car | ID3 + chi2 99% - ID3 (gain) | -6.0886 | 0 | 0 | 20 | -8.5874 | 0.0 |
| car | ID3 (gain ratio) - ID3 (gain) | 0.6069 | 14 | 3 | 3 | 0.8973 | 0.3808 |
| tictactoe | ID3 subsets (Sec. 7) - ID3 (gain) | 8.4028 | 20 | 0 | 0 | 4.0112 | 0.0007 |
| tictactoe | ID3 subsets (Sec. 7) - CART (sklearn) | -0.2778 | 7 | 5 | 8 | -0.4149 | 0.6829 |
| tictactoe | ID3 (gain) - CART (sklearn) | -8.6806 | 0 | 0 | 20 | -3.7562 | 0.0013 |
| tictactoe | ID3 + chi2 99% - ID3 (gain) | -8.316 | 0 | 0 | 20 | -3.0911 | 0.006 |
| tictactoe | ID3 (gain ratio) - ID3 (gain) | 1.4236 | 16 | 4 | 0 | 1.495 | 0.1513 |
| bcl | ID3 subsets (Sec. 7) - ID3 (gain) | 0.2907 | 8 | 3 | 9 | 0.0954 | 0.925 |
| bcl | ID3 subsets (Sec. 7) - CART (sklearn) | -1.3953 | 9 | 1 | 10 | -0.4764 | 0.6393 |
| bcl | ID3 (gain) - CART (sklearn) | -1.686 | 5 | 3 | 12 | -0.5 | 0.6228 |
| bcl | ID3 + chi2 99% - ID3 (gain) | 6.686 | 17 | 2 | 1 | 2.271 | 0.035 |
| bcl | ID3 (gain ratio) - ID3 (gain) | 2.4419 | 14 | 0 | 6 | 0.6908 | 0.498 |
| wdbc | ID3 (gain) - CART (sklearn) | -0.0 | 7 | 6 | 7 | -0.0 | 1.0 |
| wdbc | ID3 + chi2 99% - ID3 (gain) | 0.2047 | 9 | 8 | 3 | 0.3548 | 0.7267 |
| wdbc | ID3 (gain ratio) - ID3 (gain) | -1.345 | 2 | 1 | 17 | -1.4047 | 0.1763 |

## Chess learning curve (ID3 with gain; fraction of the data used for training)

| train_frac | accuracy | acc_se | nodes |
|---|---|---|---|
| 0.02 | 89.67 | 1.02 | 18.7 |
| 0.05 | 93.81 | 0.3 | 30.25 |
| 0.1 | 95.5 | 0.19 | 41.75 |
| 0.2 | 97.67 | 0.15 | 57.25 |

| train_frac | accuracy | acc_se | nodes |
|---|---|---|---|
| 0.4 | 98.73 | 0.11 | 76.75 |
| 0.7 | 99.37 | 0.08 | 86.95 |

## E3. Depth control and the χ² level α (real data)




| dataset | control | value | train_acc | test_acc | test_se | nodes |
|---|---|---|---|---|---|---|
| bcl | alpha | 0.0001 | 72.2 | 70.4 | 0.5 | 3.3 |
| bcl | alpha | 0.001 | 72.7 | 69.9 | 0.4 | 5.0 |
| bcl | alpha | 0.01 | 74.5 | 69.6 | 0.8 | 8.3 |
| bcl | alpha | 0.05 | 78.4 | 69.8 | 0.9 | 25.5 |
| bcl | alpha | 0.1 | 81.8 | 69.0 | 0.8 | 48.85 |
| bcl | alpha | 0.2 | 88.8 | 66.4 | 0.9 | 101.9 |
| bcl | alpha | 0.5 | 97.7 | 65.5 | 1.0 | 222.0 |
| bcl | alpha | none | 98.3 | 65.3 | 1.0 | 230.7 |
| bcl | depth | 0 | 70.5 | 69.8 | 0.0 | 1.0 |
| bcl | depth | 1 | 72.1 | 68.9 | 0.7 | 7.9 |
| bcl | depth | 2 | 79.2 | 68.3 | 0.7 | 44.05 |
| bcl | depth | 3 | 87.1 | 67.5 | 0.9 | 117.2 |
| bcl | depth | 4 | 94.0 | 65.6 | 0.9 | 183.1 |
| bcl | depth | 5 | 97.1 | 65.4 | 0.9 | 218.65 |
| bcl | depth | 6 | 98.2 | 65.4 | 1.0 | 229.45 |
| bcl | depth | 7 | 98.3 | 65.3 | 1.0 | 230.7 |
| bcl | depth | 8 | 98.3 | 65.3 | 1.0 | 230.7 |
| bcl | depth | full | 98.3 | 65.3 | 1.0 | 230.7 |
| car | alpha | 0.0001 | 87.2 | 84.2 | 0.3 | 48.0 |
| car | alpha | 0.001 | 89.2 | 84.9 | 0.2 | 64.15 |
| car | alpha | 0.01 | 91.0 | 86.4 | 0.3 | 88.05 |
| car | alpha | 0.05 | 94.4 | 89.1 | 0.3 | 135.5 |
| car | alpha | 0.1 | 95.8 | 90.9 | 0.3 | 159.05 |
| car | alpha | 0.2 | 97.2 | 92.9 | 0.2 | 204.7 |
| car | alpha | 0.5 | 100.0 | 92.8 | 0.2 | 337.8 |
| car | alpha | none | 100.0 | 92.7 | 0.2 | 338.1 |
| car | depth | 0 | 70.1 | 69.9 | 0.0 | 1.0 |
| car | depth | 1 | 70.1 | 69.9 | 0.0 | 4.0 |
| car | depth | 2 | 78.1 | 76.8 | 0.2 | 10.0 |
| car | depth | 3 | 82.5 | 79.9 | 0.3 | 25.85 |
| car | depth | 4 | 90.8 | 86.2 | 0.2 | 87.6 |
| car | depth | 5 | 97.0 | 92.7 | 0.2 | 210.65 |
| car | depth | 6 | 100.0 | 92.7 | 0.2 | 338.1 |
| car | depth | 7 | 100.0 | 92.7 | 0.2 | 338.1 |
| car | depth | 8 | 100.0 | 92.7 | 0.2 | 338.1 |
| car | depth | full | 100.0 | 92.7 | 0.2 | 338.1 |
| chess | alpha | 0.0001 | 98.6 | 98.3 | 0.2 | 42.05 |
| chess | alpha | 0.001 | 99.3 | 99.0 | 0.1 | 51.35 |
| chess | alpha | 0.01 | 99.6 | 99.2 | 0.1 | 59.95 |

| dataset | control | value | train_acc | test_acc | test_se | nodes |
|---|---|---|---|---|---|---|
| chess | alpha | 0.05 | 99.7 | 99.2 | 0.1 | 67.05 |
| chess | alpha | 0.1 | 99.8 | 99.3 | 0.1 | 74.05 |
| chess | alpha | 0.2 | 100.0 | 99.3 | 0.1 | 80.7 |
| chess | alpha | 0.5 | 100.0 | 99.3 | 0.1 | 84.1 |
| chess | alpha | none | 100.0 | 99.3 | 0.1 | 85.6 |
| chess | depth | 0 | 52.2 | 52.2 | 0.0 | 1.0 |
| chess | depth | 1 | 66.0 | 66.2 | 0.2 | 3.0 |
| chess | depth | 2 | 75.6 | 75.2 | 0.2 | 5.0 |
| chess | depth | 3 | 90.4 | 90.4 | 0.2 | 7.0 |
| chess | depth | 4 | 93.8 | 93.6 | 0.3 | 11.0 |
| chess | depth | 5 | 94.1 | 94.0 | 0.1 | 18.2 |
| chess | depth | 6 | 94.4 | 94.0 | 0.2 | 26.1 |
| chess | depth | 7 | 96.9 | 96.6 | 0.2 | 36.6 |
| chess | depth | 8 | 98.5 | 98.3 | 0.1 | 47.0 |
| chess | depth | full | 100.0 | 99.3 | 0.1 | 85.6 |
| votes | alpha | 0.0001 | 95.4 | 96.1 | 0.3 | 5.6 |
| votes | alpha | 0.001 | 95.4 | 96.1 | 0.3 | 6.9 |
| votes | alpha | 0.01 | 96.1 | 95.3 | 0.4 | 10.9 |
| votes | alpha | 0.05 | 97.2 | 95.0 | 0.4 | 15.7 |
| votes | alpha | 0.1 | 97.9 | 95.0 | 0.4 | 21.2 |
| votes | alpha | 0.2 | 98.6 | 94.5 | 0.4 | 29.5 |
| votes | alpha | 0.5 | 98.9 | 94.4 | 0.4 | 34.5 |
| votes | alpha | none | 98.9 | 94.4 | 0.4 | 34.5 |
| votes | depth | 0 | 61.3 | 61.5 | 0.0 | 1.0 |
| votes | depth | 1 | 95.4 | 96.1 | 0.3 | 3.0 |
| votes | depth | 2 | 95.4 | 96.1 | 0.3 | 6.9 |
| votes | depth | 3 | 96.3 | 95.4 | 0.4 | 12.7 |
| votes | depth | 4 | 97.5 | 94.6 | 0.4 | 20.9 |
| votes | depth | 5 | 98.2 | 94.4 | 0.4 | 28.1 |
| votes | depth | 6 | 98.8 | 94.5 | 0.4 | 32.1 |
| votes | depth | 7 | 98.9 | 94.4 | 0.4 | 33.8 |
| votes | depth | 8 | 98.9 | 94.4 | 0.4 | 34.3 |
| votes | depth | full | 98.9 | 94.4 | 0.4 | 34.5 |

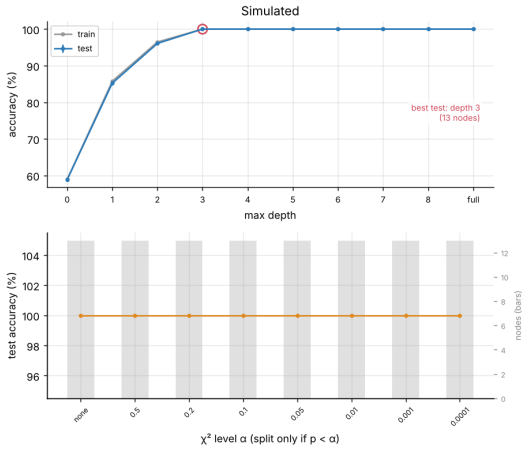
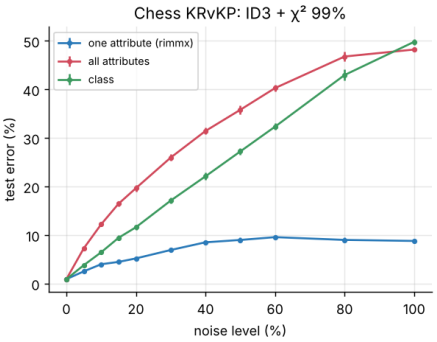
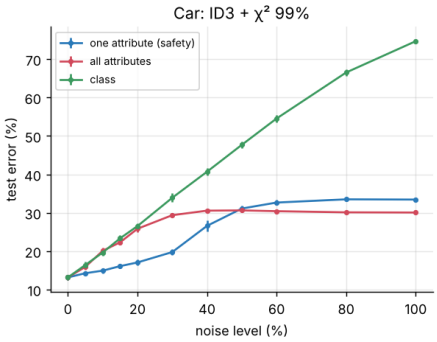
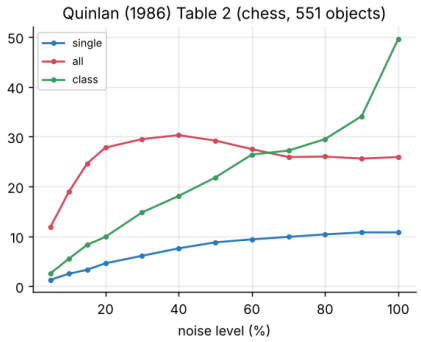



Figure A3b. The same study on the simulated tree (true depth 3, clean data): the best depth is the true depth.

## E4. Noise on real data: the paper's Table 2 (test error %, ID3 + $\chi^2$ 99 %)




| noise level | car · all | car · class | car · single | chess · all | chess · class | chess · single |
|---|---|---|---|---|---|---|
| 0 % | 13.2 | 13.2 | 13.2 | 0.9 | 0.9 | 0.9 |
| 5 % | 15.9 | 16.4 | 14.3 | 7.3 | 3.8 | 2.5 |
| 10 % | 20.1 | 19.6 | 14.9 | 12.2 | 6.4 | 4.0 |
| 15 % | 22.3 | 23.3 | 16.1 | 16.4 | 9.4 | 4.5 |
| 20 % | 25.8 | 26.5 | 17.1 | 19.7 | 11.6 | 5.2 |
| 30 % | 29.4 | 33.9 | 19.8 | 26.0 | 17.1 | 6.9 |
| 40 % | 30.6 | 40.7 | 26.6 | 31.4 | 22.1 | 8.5 |
| 50 % | 30.6 | 47.7 | 31.1 | 35.8 | 27.2 | 9.0 |
| 60 % | 30.4 | 54.6 | 32.7 | 40.3 | 32.3 | 9.6 |
| 80 % | 30.1 | 66.5 | 33.5 | 46.8 | 43.0 | 9.0 |
| 100 % | 30.1 | 74.7 | 33.4 | 48.2 | 49.8 | 8.8 |

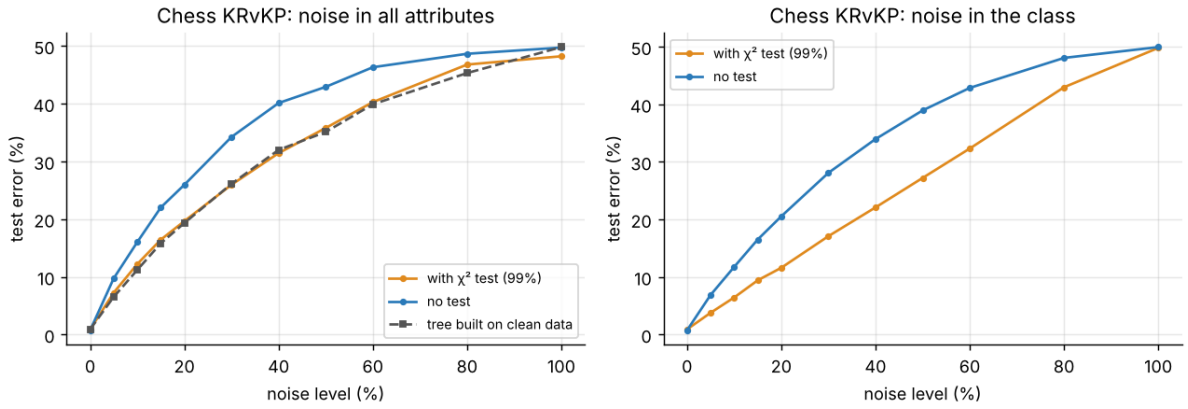



Figure A4. $\chi^2$ test on and off, and a tree built on clean data classifying noisy objects.

## Mean tree size (nodes) under noise, with and without the $\chi^2$ test

| level | car·all·none | car·all·$\chi^2$ | car·class·none | car·class·$\chi^2$ | car·single·none | car·single·$\chi^2$ | chess·all·none | chess·all·$\chi^2$ | chess·class·none | chess·class·$\chi^2$ | chess·single·none | chess·single·$\chi^2$ |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0.00 | 340.0 | 90.0 | 340.0 | 90.0 | 340.0 | 90.0 | 85.0 | 60.0 | 85.0 | 60.0 | 85.0 | 60.0 |
| 0.05 | 562.0 | 75.0 | 630.0 | 79.0 | 425.0 | 87.0 | 489.0 | 103.0 | 455.0 | 63.0 | 206.0 | 77.0 |
| 0.1 | 703.0 | 60.0 | 807.0 | 72.0 | 482.0 | 84.0 | 708.0 | 101.0 | 709.0 | 65.0 | 287.0 | 85.0 |
| 0.15 | 778.0 | 50.0 | 973.0 | 66.0 | 537.0 | 81.0 | 851.0 | 92.0 | 954.0 | 57.0 | 336.0 | 96.0 |
| 0.2 | 849.0 | 37.0 | 1104.0 | 59.0 | 568.0 | 75.0 | 932.0 | 84.0 | 1160.0 | 53.0 | 368.0 | 103.0 |
| 0.3 | 942.0 | 27.0 | 1314.0 | 45.0 | 655.0 | 73.0 | 1076.0 | 68.0 | 1486.0 | 40.0 | 443.0 | 104.0 |
| 0.4 | 1018.0 | 22.0 | 1469.0 | 34.0 | 705.0 | 68.0 | 1168.0 | 53.0 | 1746.0 | 32.0 | 503.0 | 107.0 |
| 0.5 | 1072.0 | 17.0 | 1597.0 | 25.0 | 737.0 | 59.0 | 1229.0 | 42.0 | 1939.0 | 24.0 | 535.0 | 104.0 |
| 0.6 | 1114.0 | 14.0 | 1691.0 | 16.0 | 776.0 | 52.0 | 1260.0 | 35.0 | 2100.0 | 21.0 | 562.0 | 106.0 |
| 0.8 | 1162.0 | 6.0 | 1803.0 | 4.0 | 813.0 | 46.0 | 1320.0 | 12.0 | 2308.0 | 11.0 | 561.0 | 108.0 |
| 1.0 | 1161.0 | 2.0 | 1830.0 | 1.0 | 820.0 | 45.0 | 1330.0 | 2.0 | 2354.0 | 2.0 | 569.0 | 106.0 |

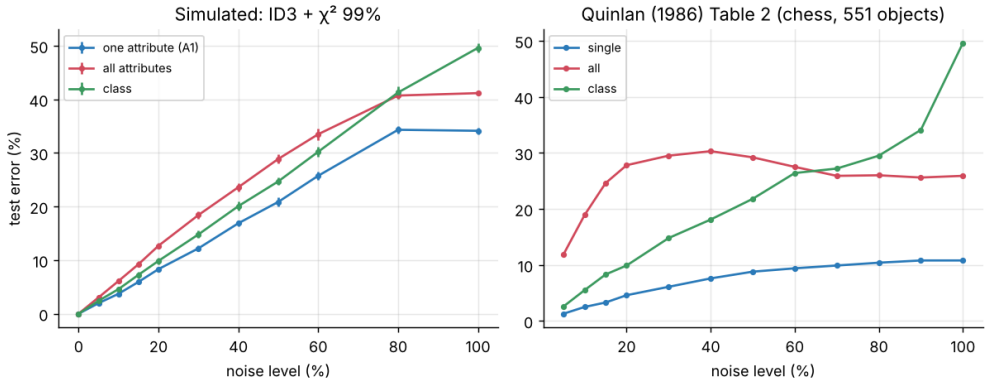



Figure A4b. Sanity check: the same noise study on simulated data.

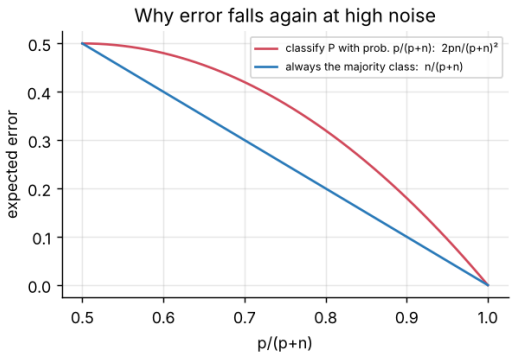
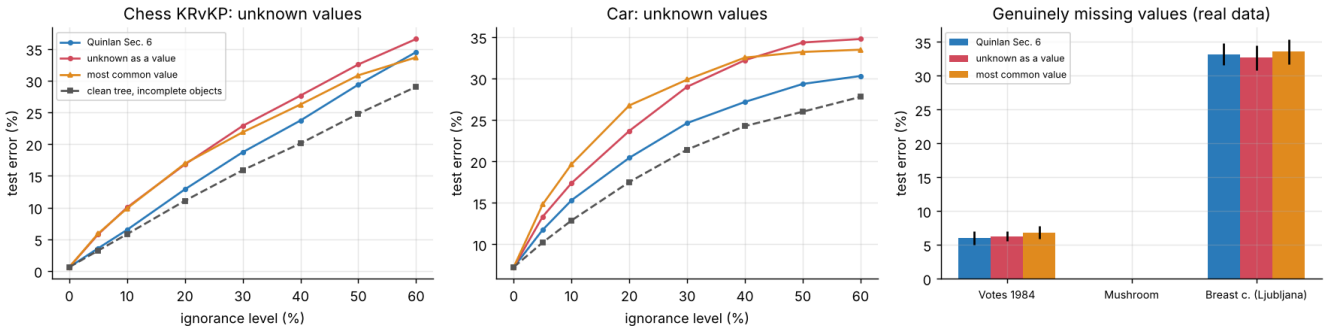



Figure A5. Section 5's explanation of the falling error at high noise.

## E5. Unknown attribute values (test error %, Section 6)




| dataset | level | error: Quinlan Sec. 6 | error: unknown as a value | error: most common value | error: clean tree, incomplete objects |
|---|---|---|---|---|---|
| car | 0.0 | 7.2 | 7.2 | 7.2 | 7.2 |
| car | 0.05 | 11.7 | 13.3 | 14.8 | 10.2 |
| car | 0.1 | 15.3 | 17.4 | 19.7 | 12.9 |
| car | 0.2 | 20.4 | 23.7 | 26.8 | 17.5 |
| car | 0.3 | 24.6 | 29.0 | 29.9 | 21.4 |
| car | 0.4 | 27.2 | 32.2 | 32.5 | 24.3 |
| car | 0.5 | 29.3 | 34.3 | 33.2 | 26.0 |
| car | 0.6 | 30.3 | 34.8 | 33.5 | 27.8 |
| chess | 0.0 | 0.6 | 0.6 | 0.6 | 0.6 |
| chess | 0.05 | 3.6 | 5.8 | 5.9 | 3.2 |
| chess | 0.1 | 6.5 | 10.0 | 9.9 | 5.8 |
| chess | 0.2 | 12.9 | 16.8 | 16.9 | 11.1 |
| chess | 0.3 | 18.7 | 22.9 | 21.9 | 15.9 |
| chess | 0.4 | 23.7 | 27.6 | 26.2 | 20.1 |
| chess | 0.5 | 29.4 | 32.6 | 30.8 | 24.8 |
| chess | 0.6 | 34.5 | 36.6 | 33.7 | 29.0 |

## Genuinely missing values (accuracy, 20 splits)

| dataset | Quinlan Sec. 6 (mean) | Quinlan Sec. 6 (se) | unknown as a value (mean) | unknown as a value (se) | most common value (mean) | most common value (se) |
|---|---|---|---|---|---|---|
| votes | 0.94 | 0.0049 | 0.9377 | 0.0037 | 0.9319 | 0.0049 |
| mushroom | 1.0 | 0.0 | 1.0 | 0.0 | 1.0 | 0.0 |
| bcl | 0.668 | 0.0082 | 0.6733 | 0.0095 | 0.6645 | 0.0092 |

## E6. Gain bias towards many-valued attributes (Section 7, simulated id_code)

| criterion | id_code_tests | accuracy | acc_se | nodes |
|---|---|---|---|---|
| gain | 1.0 | 0.9553 | 0.0019 | 171.45 |
| gain ratio | 0.0 | 1.0 | 0.0 | 13.0 |
| gain, binary subsets | 1.0 | 0.9772 | 0.0037 | 12.7 |

| attribute | values | gain | IV | gain_ratio |
|---|---|---|---|---|
| A1 | 3 | 0.5501 | 1.5833 | 0.3474 |
| id_code | 142 | 0.3268 | 6.9471 | 0.047 |
| A2 | 3 | 0.1049 | 1.5818 | 0.0663 |
| A4 | 3 | 0.0083 | 1.5825 | 0.0053 |
| A6 | 3 | 0.0043 | 1.5813 | 0.0027 |
| A3 | 3 | 0.0022 | 1.5848 | 0.0014 |
| A7 | 3 | 0.0015 | 1.5841 | 0.0009 |
| A8 | 3 | 0.0008 | 1.5827 | 0.0005 |
| A10 | 3 | 0.0006 | 1.5817 | 0.0004 |
| A9 | 3 | 0.0005 | 1.5842 | 0.0003 |

| attribute | values | gain | IV | gain_ratio |
|---|---|---|---|---|
| A5 | 3 | 0.0 | 1.5848 | 0.0 |

## E7. Windowing on chess and mushroom (Section 4)

| dataset | method | objects_used | nodes | train_errors | test_accuracy | seconds |
|---|---|---|---|---|---|---|
| chess | direct | 2237 | 85 | 0 | 0.9885 | 0.2204 |
| chess | windowing (100 + 50 per round) | 304 | 76 | 0 | 0.9896 | 0.8718 |
| mushroom | direct | 5687 | 38 | 0 | 1.0 | 0.0852 |
| mushroom | windowing (100 + 50 per round) | 288 | 37 | 0 | 1.0 | 0.4325 |

| iteration | window | errors | dataset |
|---|---|---|---|
| 0 | 100 | 217 | chess |
| 1 | 150 | 224 | chess |
| 2 | 200 | 80 | chess |
| 3 | 250 | 33 | chess |
| 4 | 283 | 17 | chess |
| 5 | 300 | 4 | chess |
| 6 | 304 | 0 | chess |
| 0 | 100 | 166 | mushroom |
| 1 | 150 | 58 | mushroom |
| 2 | 200 | 99 | mushroom |
| 3 | 250 | 38 | mushroom |
| 4 | 288 | 0 | mushroom |

# D. Code listing

## id3.py

```
"""
id3.py -- a from-scratch implementation of Quinlan's ID3 (Machine Learning 1:81-106, 1986).

Everything the paper describes is implemented with NumPy/pandas only:

  Section 4  entropy I(p, n), expected information E(A), gain(A); divide-and-conquer tree;
             empty branch -> 'null' leaf (original ID3) or majority class of the parent;
             the windowing loop.
  Section 5  noise: majority-class leaves for inadequate attributes, and the chi-square
             test of independence that refuses to split on an attribute whose irrelevance
             cannot be rejected (e.g. at 99 % confidence).
  Section 6  unknown attribute values: proportional redistribution when computing gain,
             unknowns discarded below the chosen split, and 'token' splitting at prediction.
  Section 7  gain ratio = gain / IV(A), chosen among attributes with at least average gain.

  Section 7  also the binary value-subset tests (ASSISTANT) and thresholds for continuous
             attributes at midpoints of consecutive values (split='subset', continuous=[...]).

Extra knobs that are not in the paper (labelled as such): max_depth and min_samples_split.
These are the "depth control" studied in the project.

The paper uses two classes (P, N); here every formula is written for K >= 2 classes,
which reduces to the paper's formulas when K = 2.
"""
from __future__ import annotations

from dataclasses import dataclass, field
from typing import Dict, List, Optional

import numpy as np
import pandas as pd
from scipy.stats import chi2 as _chi2

UNKNOWN_TOKENS = {"?", "", "nan", "NaN", "None"}


# ---------------------------------------------------------------------------------------
# Information measures (Section 4 and 7)
# ---------------------------------------------------------------------------------------
def entropy(counts) -> float:
    """I(p, n) generalised to K classes: -sum_k (c_k/N) log2 (c_k/N), with 0 log 0 = 0."""
    c = np.asarray(counts, dtype=float)
    total = c.sum()
    if total <= 0:
        return 0.0
    q = c[c > 0] / total
    return float(-(q * np.log2(q)).sum())


def expected_information(table) -> float:
    """E(A) = sum_i (|C_i|/|C|) I(C_i) for a values x classes count table."""
    t = np.asarray(table, dtype=float)
    total = t.sum()
    if total <= 0:
        return 0.0
    return float(sum(row.sum() / total * entropy(row) for row in t))


def information_gain(table) -> float:
    """gain(A) = I(C) - E(A)."""
    t = np.asarray(table, dtype=float)
    return entropy(t.sum(axis=0)) - expected_information(t)


def intrinsic_value(table) -> float:
    """IV(A) = -sum_i (|C_i|/|C|) log2 (|C_i|/|C|), the information in the attribute's value."""
    return entropy(np.asarray(table, dtype=float).sum(axis=1))


def gain_ratio(table) -> float:
    iv = intrinsic_value(table)
    return information_gain(table) / iv if iv > 0 else 0.0


def chi_square(table):
    """Quinlan's Section-5 statistic sum_i sum_k (c_ik - c'_ik)^2 / c'_ik and its p-value.

    c'_ik = c_k * |C_i| / |C| is the count expected if A were irrelevant to the class.
    Degrees of freedom (v-1)(K-1); the paper's two-class case gives v-1.
    """
    t = np.asarray(table, dtype=float)
    t = t[t.sum(axis=1) > 0]            # values that actually occur
    t = t[:, t.sum(axis=0) > 0]         # classes that actually occur
    if t.shape[0] < 2 or t.shape[1] < 2:
        return 0.0, 1.0
    expected = np.outer(t.sum(axis=1), t.sum(axis=0)) / t.sum()
    stat = float(((t - expected) ** 2 / expected).sum())
```

```
    dof = (t.shape[0] - 1) * (t.shape[1] - 1)
    return stat, float(_chi2.sf(stat, dof))


# ---------------------------------------------------------------------------------------
# Tree structure
# ---------------------------------------------------------------------------------------
@dataclass
class Node:
    dist: np.ndarray                     # class counts that reached this node in training
    attribute: Optional[str] = None      # None -> leaf
    kind: str = "value"                  # 'value' (multiway), 'subset' (binary, Section 7) or 'threshold'
    param: object = None                 # frozenset of values for 'subset', cut point for 'threshold'
    children: Dict[str, "Node"] = field(default_factory=dict)
    ratios: Dict[str, float] = field(default_factory=dict)  # |C_i|/sum|C_j| for token splitting
    is_null: bool = False                # an empty-branch leaf in 'null' mode
    gain: float = 0.0
    p_value: float = 1.0

    @property
    def is_leaf(self) -> bool:
        return self.attribute is None

    def branch_of(self, v):
        """Branch key for a known value v, or None if the value was never seen here."""
        if self.kind == "value":
            return v if v in self.children else None
        if self.kind == "subset":
            return "in" if v in self.param else ("out" if v in self._out else None)
        return "le" if float(v) <= self.param else "gt"

    def test_text(self, key) -> str:
        if self.kind == "value":
            return f"{self.attribute} = {key}"
        if self.kind == "subset":
            vals = ",".join(sorted(self.param)) if key == "in" else ",".join(sorted(self._out))
            return f"{self.attribute} in {{{vals}}}"
        return f"{self.attribute} {'<=' if key == 'le' else '>'} {self.param:.4g}"


def _is_unknown(series: pd.Series) -> np.ndarray:
    s = series.astype(object)
    return (s.isna() | s.astype(str).isin(UNKNOWN_TOKENS)).to_numpy()


def _adjust(table, unknown_counts):
    """Section 6: spread the unknown objects over the outcomes in proportion to |C_i|."""
    if unknown_counts.sum() == 0 or table.sum() == 0:
        return table
    ratio = table.sum(axis=1) / table.sum()
    return table + np.outer(ratio, unknown_counts)


class ID3:
    """ID3 decision tree.

    Parameters
    ----------
    criterion : 'gain' (Section 4) or 'gain_ratio' (Section 7).
    chi2_alpha : None disables the Section-5 test. Otherwise an attribute is only tested when
        the chi-square p-value is below alpha (alpha = 0.01 is the paper's 99 % confidence).
    split : 'multiway' -- one branch per value (ID3, Section 4);
            'subset'   -- the binary value-subset tests of ASSISTANT discussed in Section 7.
              All 2^(v-1)-1 subsets are searched when v <= max_subset_values; above that, values are
              ordered by their class-1 proportion and only the v-1 cuts of that order are tried,
              which is exact for two classes (Breiman et al. 1984) and a heuristic otherwise.
    continuous : names of numeric attributes, split by a threshold at the midpoint of two
        consecutive observed values (Section 7). Binary tests may reuse an attribute below them.
    max_depth, min_samples_split : depth control (not in the paper; added for the project).
    empty_branch : 'majority' (the paper's recommended fix) or 'null' (original ID3).
    """

    def __init__(self, criterion: str = "gain", chi2_alpha: Optional[float] = None,
                 max_depth: Optional[int] = None, min_samples_split: int = 2,
                 empty_branch: str = "majority", split: str = "multiway",
                 continuous: Optional[List[str]] = None, max_subset_values: int = 10):
        assert criterion in ("gain", "gain_ratio")
        assert empty_branch in ("majority", "null")
        assert split in ("multiway", "subset")
        self.criterion = criterion
        self.chi2_alpha = chi2_alpha
        self.max_depth = max_depth
        self.min_samples_split = min_samples_split
        self.empty_branch = empty_branch
        self.split = split
        self.continuous = list(continuous or [])
        self.max_subset_values = max_subset_values

    # ------------------------------------------------------------------ fitting
    def fit(self, X: pd.DataFrame, y) -> "ID3":
        X = X.astype(object).reset_index(drop=True)
        y = pd.Series(np.asarray(y)).astype(str).reset_index(drop=True)
        self.classes_ = np.array(sorted(y.unique()))
```

```
        self.attributes_ = list(X.columns)
        self.domains_, self._X = {}, {}
        for a in X.columns:
            unk = _is_unknown(X[a])
            if a in self.continuous:
                col = pd.to_numeric(X[a].where(~unk), errors="coerce").to_numpy(dtype=float)
                self._X[a] = col                                  # NaN marks 'unknown'
                self.domains_[a] = None
            else:
                col = X[a].astype(str).to_numpy(dtype=object).copy()  # never write into the caller's frame
                col[unk] = None
                self._X[a] = col                                  # None marks 'unknown'
                # The domain of each attribute ("values that could have appeared")
                self.domains_[a] = sorted(set(col[~unk]))
        y_idx = np.searchsorted(self.classes_, y.to_numpy())
        self.root_ = self._build(np.arange(len(y_idx)), y_idx, list(X.columns), depth=0)
        del self._X
        return self

    def _counts(self, y_idx):
        return np.bincount(y_idx, minlength=len(self.classes_)).astype(float)

    def _score(self, table, unknown_counts):
        adjusted = _adjust(table, unknown_counts)
        g = information_gain(adjusted)
        if self.criterion == "gain":
            score = g
        else:
            iv = intrinsic_value(adjusted)
            score = g / iv if iv > 0 else 0.0
        return score, g, chi_square(table)[1]

    def _best_test(self, rows, y_idx, a):
        """Best test on attribute a: dict(score, gain, p, kind, param) or None."""
        K = len(self.classes_)
        col = self._X[a][rows]
        if a in self.continuous:
            known = ~np.isnan(col)
            unknown_counts = np.bincount(y_idx[~known], minlength=K).astype(float)
            x, c = col[known], y_idx[known]
            if len(x) < 2:
                return None
            order = np.argsort(x, kind="stable")
            x, c = x[order], c[order]
            onehot = np.eye(K)[c]
            left = np.cumsum(onehot, axis=0)[:-1]                 # counts with x <= cut
            total = left[-1] + onehot[-1]
            cuts = np.where(np.diff(x) > 0)[0]                     # between distinct values
            if len(cuts) == 0:
                return None
            # Vectorised over all candidate cuts: 2 x K tables, Section-6 adjustment, gain and IV
            L, R = left[cuts], total - left[cuts]
            nL, nR = L.sum(1, keepdims=True), R.sum(1, keepdims=True)
            n = nL + nR
            u = unknown_counts[None, :]
            La, Ra = L + nL / n * u, R + nR / n * u
            N = La.sum(1) + Ra.sum(1)

            def H(M):
                tot = M.sum(1, keepdims=True)
                q = np.where(M > 0, M / np.where(tot > 0, tot, 1), 1.0)
                return -(q * np.log2(q)).sum(1)

            I0 = entropy(total + unknown_counts)
            gains = I0 - (La.sum(1) / N * H(La) + Ra.sum(1) / N * H(Ra))
            if self.criterion == "gain":
                scores = gains
            else:
                pl = (nL / n).ravel()
                iv = -(pl * np.log2(pl) + (1 - pl) * np.log2(1 - pl))
                scores = np.where(iv > 0, gains / iv, 0.0)
            j = int(np.argmax(scores))
            i = cuts[j]
            p = chi_square(np.vstack([L[j], R[j]]))[1]
            best = dict(score=float(scores[j]), gain=float(gains[j]), p=p, kind="threshold",
                        param=(x[i] + x[i + 1]) / 2)
            return best

        known = np.array([v is not None for v in col])
        values = self.domains_[a]
        pos = {v: i for i, v in enumerate(values)}
        table = np.zeros((len(values), K))
        for v, c in zip(col[known], y_idx[known]):
            table[pos[v], c] += 1
        unknown_counts = np.bincount(y_idx[~known], minlength=K).astype(float)
        if table.sum() == 0:
            return None
        if self.split == "multiway":
            score, g, p = self._score(table, unknown_counts)
            return dict(score=score, gain=g, p=p, kind="value", param=None)

        present = [i for i in range(len(values)) if table[i].sum() > 0]
        if len(present) < 2:
```

```
            return None
        if len(present) <= self.max_subset_values:
            first, rest = present[0], present[1:]
            candidates = []
            for mask in range(2 ** len(rest)):                    # subsets that contain `first`
                S = [first] + [rest[j] for j in range(len(rest)) if mask >> j & 1]
                if len(S) < len(present):
                    candidates.append(S)
        else:
            frac = table[present, -1] / table[present].sum(axis=1)
            ordered = [present[i] for i in np.argsort(frac, kind="stable")]
            candidates = [ordered[:k] for k in range(1, len(ordered))]
        best = None
        for S in candidates:
            inside = table[S].sum(axis=0)
            t2 = np.vstack([inside, table.sum(axis=0) - inside])
            score, g, p = self._score(t2, unknown_counts)
            if best is None or score > best["score"] + 1e-12:
                best = dict(score=score, gain=g, p=p, kind="subset",
                            param=frozenset(values[i] for i in S))
        return best

    def _build(self, rows, y_idx_all, attributes, depth) -> Node:
        y_idx = y_idx_all[rows]
        dist = self._counts(y_idx)
        node = Node(dist=dist)
        if (dist > 0).sum() <= 1 or not attributes:
            return node                                   # pure, or attributes exhausted
        if self.max_depth is not None and depth >= self.max_depth:
            return node
        if len(rows) < self.min_samples_split:
            return node

        scored = []
        for a in attributes:
            t = self._best_test(rows, y_idx, a)
            if t is None or not np.isfinite(t["score"]):
                continue
            if self.chi2_alpha is not None and t["p"] >= self.chi2_alpha:
                continue                                  # Section 5: irrelevance not rejected
            scored.append((a, t))
        if not scored:
            return node                                   # majority-class leaf (Section 5)
        if self.criterion == "gain_ratio":
            # Section 7: maximise the ratio among attributes with at least average gain
            avg_gain = np.mean([t["gain"] for _, t in scored])
            scored = [s for s in scored if s[1]["gain"] >= avg_gain - 1e-12]
        best_a, best = max(scored, key=lambda s: (s[1]["score"], -self.attributes_.index(s[0])))
        if best["gain"] <= 1e-12:
            return node                                   # no split separates anything

        node.attribute, node.gain, node.p_value = best_a, best["gain"], best["p"]
        node.kind, node.param = best["kind"], best["param"]
        col = self._X[best_a][rows]
        if node.kind == "threshold":
            known = ~np.isnan(col)
            branches = {"le": known & (np.nan_to_num(col, nan=np.inf) <= node.param),
                        "gt": known & (np.nan_to_num(col, nan=-np.inf) > node.param)}
            remaining = attributes                        # a numeric attribute can be cut again
        else:
            known = np.array([v is not None for v in col])
            if node.kind == "value":
                branches = {v: known & (col == v) for v in self.domains_[best_a]}
                remaining = [a for a in attributes if a != best_a]
            else:
                inside = np.array([v in node.param for v in col])
                node._out = frozenset(self.domains_[best_a]) - node.param
                branches = {"in": known & inside, "out": known & ~inside}
                remaining = attributes                    # a subset test can be refined below
        n_known = known.sum()
        for key, m in branches.items():
            sub = rows[m]                                 # unknowns are discarded (Section 6)
            node.ratios[key] = len(sub) / n_known if n_known else 0.0
            if len(sub) == 0:
                node.children[key] = Node(dist=dist.copy(), is_null=(self.empty_branch == "null"))
            elif len(sub) == len(rows):
                node.children[key] = Node(dist=dist.copy())   # cannot happen for gain > 0; guard
            else:
                node.children[key] = self._build(sub, y_idx_all, remaining, depth + 1)
        return node

    # ------------------------------------------------------------------ prediction
    def _leaf_vote(self, node: Node, token: float) -> np.ndarray:
        if node.is_null:
            return np.zeros(len(self.classes_))           # 'null' fails to classify
        return token * node.dist / node.dist.sum()

    def _route(self, node: Node, row: dict, token: float) -> np.ndarray:
        if node.is_leaf:
            return self._leaf_vote(node, token)
        v = row.get(node.attribute)
        unknown = v is None or (isinstance(v, float) and np.isnan(v)) or str(v) in UNKNOWN_TOKENS
        key = None
```

```
        if not unknown:
            try:
                key = node.branch_of(v if node.kind == "threshold" else str(v))
            except (TypeError, ValueError):
                key = None
        if key is not None:
            return self._route(node.children[key], row, token)
        # Unknown (or never-seen) value: explore every branch with token * ratio_i (Section 6)
        total = np.zeros(len(self.classes_))
        for k, child in node.children.items():
            r = node.ratios.get(k, 0.0)
            if r > 0:
                total += self._route(child, row, token * r)
        return total

    def predict_proba(self, X: pd.DataFrame) -> np.ndarray:
        out = []
        for row in X.astype(object).to_dict(orient="records"):
            votes = self._route(self.root_, row, 1.0)
            s = votes.sum()
            out.append(votes / s if s > 0 else np.full(len(self.classes_), np.nan))
        return np.vstack(out)

    def predict(self, X: pd.DataFrame) -> np.ndarray:
        P = self.predict_proba(X)
        pred = np.empty(len(P), dtype=object)
        ok = ~np.isnan(P).any(axis=1)
        pred[ok] = self.classes_[P[ok].argmax(axis=1)]
        pred[~ok] = "null"                                  # unclassified by a 'null' leaf
        return pred

    def score(self, X, y) -> float:
        return float(np.mean(self.predict(X) == np.asarray(y).astype(str)))

    # ------------------------------------------------------------------ inspection
    def _walk(self, node=None):
        node = node or self.root_
        yield node
        for c in node.children.values():
            yield from self._walk(c)

    @property
    def n_nodes(self) -> int:
        return sum(1 for _ in self._walk())

    @property
    def n_leaves(self) -> int:
        return sum(1 for n in self._walk() if n.is_leaf)

    @property
    def depth(self) -> int:
        def d(n):
            return 0 if n.is_leaf else 1 + max(d(c) for c in n.children.values())
        return d(self.root_)

    def export_text(self, node: Node = None, indent: str = "") -> str:
        node = node or self.root_
        lines = []
        if node.is_leaf:
            label = "null" if node.is_null else self.classes_[node.dist.argmax()]
            return f"{indent}-> {label}  " + ", ".join(f"{c}: {int(k)}" for c, k in zip(self.classes_, node.dist))
        for v, child in node.children.items():
            if child.is_leaf:
                lab = "null" if child.is_null else self.classes_[child.dist.argmax()]
                lines.append(f"{indent}{node.test_text(v)}: {lab}  (n={int(child.dist.sum())})")
            else:
                lines.append(f"{indent}{node.test_text(v)}:")
                lines.append(self.export_text(child, indent + "|   "))
        return "\n".join(lines)


# ---------------------------------------------------------------------------------------
# Windowing (Section 4)
# ---------------------------------------------------------------------------------------
def fit_windowed(model_factory, X: pd.DataFrame, y, window: int = 100, add: int = 50,
                 max_iter: int = 50, seed: int = 0):
    """Quinlan's iterative window: grow a tree on a random window, add misclassified objects
    until the tree is correct on the whole training set (or max_iter). Returns (model, history)."""
    rng = np.random.default_rng(seed)
    y = np.asarray(y).astype(str)
    n = len(y)
    in_window = np.zeros(n, dtype=bool)
    in_window[rng.choice(n, size=min(window, n), replace=False)] = True
    history = []
    for it in range(max_iter):
        model = model_factory().fit(X[in_window], y[in_window])
        wrong = np.where(model.predict(X) != y)[0]
        history.append({"iteration": it, "window": int(in_window.sum()), "errors": len(wrong)})
        if len(wrong) == 0:
            break
        new = wrong[~in_window[wrong]]
        if len(new) == 0:
            break
```

```
        in_window[rng.choice(new, size=min(add, len(new)), replace=False)] = True
    return model, history


# ---------------------------------------------------------------------------
# Corruption used in the paper's noise and ignorance experiments (Sections 5 and 6)
# ---------------------------------------------------------------------------
def corrupt(X: pd.DataFrame, y, level: float, where: str, domains: Dict[str, List[str]],
            classes, rng, attribute: Optional[str] = None):
    """With probability `level`, replace a value by one drawn uniformly from the values that
    could have appeared (it may coincide with the true value, as in the paper's footnote 5).

    where = 'class' | 'all' | 'single' (needs `attribute`)."""
    X = X.copy()
    y = np.asarray(y).astype(object).copy()
    n = len(y)
    if where == "class":
        hit = rng.random(n) < level
        y[hit] = rng.choice(np.asarray(classes, dtype=object), size=hit.sum())
    else:
        cols = [attribute] if where == "single" else list(X.columns)
        for a in cols:
            hit = rng.random(n) < level
            col = X[a].to_numpy(dtype=object).copy()
            col[hit] = rng.choice(np.asarray(domains[a], dtype=object), size=hit.sum())
            X[a] = col
    return X, y


def blank_out(X: pd.DataFrame, level: float, rng) -> pd.DataFrame:
    """Section 6 'ignorance level': each attribute value becomes unknown with probability level."""
    values = X.to_numpy(dtype=object).copy()
    values[rng.random(values.shape) < level] = "?"
    return pd.DataFrame(values, columns=X.columns, dtype=object)
```

## simulate.py

```
"""
simulate.py -- simulated data with a KNOWN decision tree, so recovery can be checked exactly.

Ten categorical attributes A1..A10, each uniform on {a, b, c} and independent.
The class is a deterministic function of A1..A4 only (A5..A10 are irrelevant):

    A1 = a:  A2 = b -> N, otherwise P
    A1 = b:  P
    A1 = c:  A3 = c -> (A4 = a -> P, otherwise N), otherwise N

P(class = P) = 1/3*(2/3) + 1/3 + 1/3*(1/3*1/3) = 16/27 = 0.5926, so the majority-class
error rate is 11/27 = 0.4074, which is the ceiling against which noise results are read.

Optionally an 'id_code' attribute with many random values (irrelevant) is added to show the
gain criterion's bias towards many-valued attributes (paper Section 7).
"""
import numpy as np
import pandas as pd

VALUES = ["a", "b", "c"]
TRUE_TREE_TEXT = """A1 = a:
|   A2 = a: P
|   A2 = b: N
|   A2 = c: P
A1 = b: P
A1 = c:
|   A3 = a: N
|   A3 = b: N
|   A3 = c:
|   |   A4 = a: P
|   |   A4 = b: N
|   |   A4 = c: N"""
P_POSITIVE = 1 / 3 * 2 / 3 + 1 / 3 + 1 / 3 * 1 / 9


def true_class(row) -> str:
    if row["A1"] == "a":
        return "N" if row["A2"] == "b" else "P"
    if row["A1"] == "b":
        return "P"
    if row["A3"] == "c":
        return "P" if row["A4"] == "a" else "N"
    return "N"


def simulate(n: int = 2000, seed: int = 2026, n_attributes: int = 10, id_values: int = 0) -> pd.DataFrame:
    rng = np.random.default_rng(seed)
    df = pd.DataFrame({f"A{j + 1}": rng.choice(VALUES, size=n) for j in range(n_attributes)})
    df["class"] = df.apply(true_class, axis=1)
    if id_values:
        df.insert(0, "id_code", [f"v{k:03d}" for k in rng.integers(0, id_values, size=n)])
    return df


if __name__ == "__main__":
    import os
```

```
    os.makedirs("data/simulated", exist_ok=True)
    simulate(2000, seed=2026).to_csv("data/simulated/sim_tree_2000.csv", index=False)
    simulate(600, seed=7, id_values=150).to_csv("data/simulated/sim_with_id_600.csv", index=False)
    print("wrote data/simulated/sim_tree_2000.csv and data/simulated/sim_with_id_600.csv")
```

## experiments.py

```
"""
experiments.py -- every table and figure in the project, reproducible from fixed seeds.

    python experiments.py            # full run, 20 repetitions per setting (about 10 minutes on 4 cores)
    python experiments.py --quick    # 5 repetitions, for a fast check

Real public datasets carry the experiments; the simulated data (known tree) are used only as
sanity checks. Outputs go to results/*.csv and figures/*.png.
"""
from __future__ import annotations

import argparse
import os
import time
from concurrent.futures import ProcessPoolExecutor

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy import stats

import id3

HERE = os.path.dirname(os.path.abspath(__file__))
RES = os.path.join(HERE, "results")
FIG = os.path.join(HERE, "figures")
os.makedirs(RES, exist_ok=True)
os.makedirs(FIG, exist_ok=True)
plt.rcParams.update({"figure.dpi": 130, "axes.grid": True, "grid.alpha": 0.3,
                     "axes.spines.top": False, "axes.spines.right": False, "font.size": 10})
COLORS = {"single": "#2a7ab9", "all": "#d1495b", "class": "#3c9a5f", "id3": "#2a7ab9",
          "chi2": "#e08a1e", "cart": "#7a5195", "ratio": "#3c9a5f", "subset": "#d1495b"}
REPS = 20

# ---------------------------------------------------------------------------- data
# name: (file, class column, label, kind, numeric attributes?)
DATASETS = {
    "weather":  ("real/weather_quinlan1986.csv", "class", "Saturday mornings (paper Table 1)", "real", False),
    "chess":    ("real/chess_kr_vs_kp.csv", "class", "Chess endgame KR vs KP (UCI)", "real", False),
    "mushroom": ("real/mushrooms.csv", "type", "Mushroom (UCI)", "real", False),
    "votes":    ("real/house_votes_84.csv", "class", "Congressional votes 1984 (UCI)", "real", False),
    "car":      ("real/car_evaluation.csv", "class", "Car Evaluation (UCI)", "real", False),
    "tictactoe": ("real/tic_tac_toe.csv", "class", "Tic-tac-toe endgame (UCI)", "real", False),
    "bcl":      ("real/breast_cancer_ljubljana.csv", "class", "Breast cancer, Ljubljana (UCI)", "real", False),
    "wdbc":     ("real/breast_cancer_wisconsin.csv", "class", "Breast cancer, Wisconsin (UCI)", "real", True),
    "sim":      ("simulated/sim_tree_2000.csv", "class", "Simulated: known depth-3 tree", "simulated", False),
    "sim_id":   ("simulated/sim_with_id_600.csv", "class", "Simulated + 150-valued id_code", "simulated", False),
}
BENCH = ["chess", "mushroom", "votes", "car", "tictactoe", "bcl", "wdbc"]
SHORT = {"chess": "Chess KRvKP", "mushroom": "Mushroom", "votes": "Votes 1984", "car": "Car", "tictactoe": "Tic-tac-toe",
         "bcl": "Breast c. (Ljubljana)", "wdbc": "Breast c. (Wisconsin)", "sim": "Simulated"}


def load(name: str):
    f, ycol, _, _, numeric = DATASETS[name]
    d = pd.read_csv(os.path.join(HERE, "data", f), dtype=None if numeric else str, keep_default_na=False)
    X = d.drop(columns=[c for c in (ycol, "no") if c in d.columns])
    return X, d[ycol].astype(str).to_numpy()


def is_numeric(name):
    return DATASETS[name][4]


def split(X, y, test_frac, rng):
    """Stratified random split."""
    idx_tr, idx_te = [], []
    for c in np.unique(y):
        ids = rng.permutation(np.where(y == c)[0])
        k = int(round(len(ids) * test_frac))
        idx_te += list(ids[:k]); idx_tr += list(ids[k:])
    idx_tr, idx_te = np.array(idx_tr), np.array(idx_te)
    return (X.iloc[idx_tr].reset_index(drop=True), y[idx_tr],
            X.iloc[idx_te].reset_index(drop=True), y[idx_te])


def se(a):
    a = np.asarray(a, dtype=float)
    return a.std(ddof=1) / np.sqrt(len(a)) if len(a) > 1 else 0.0


def corrected_t(diff, n_train, n_test):
    """Nadeau & Bengio (2003) corrected resampled t-test for J random train/test splits:
    t = mean(d) / sqrt((1/J + n_test/n_train) * var(d)), J-1 degrees of freedom.
```

```
    The correction accounts for the overlap of training sets between splits."""
    d = np.asarray(diff, dtype=float)
    J = len(d)
    v = d.var(ddof=1)
    if v == 0:
        return (0.0, 1.0) if d.mean() == 0 else (float(np.sign(d.mean()) * np.inf), 0.0)
    t = d.mean() / np.sqrt((1 / J + n_test / n_train) * v)
    return float(t), float(2 * stats.t.sf(abs(t), J - 1))


def make(name, **kw):
    if is_numeric(name):
        X, _ = load(name)
        kw["continuous"] = list(X.columns)
    return id3.ID3(**kw)


# ---------------------------------------------------------------------------- E1
def e1_weather():
    """Reproduce the worked example of Sections 4 and 7 on Table 1."""
    X, y = load("weather")
    paper = {"outlook": (0.246, 1.578, 0.156), "temperature": (0.029, None, None),
             "humidity": (0.151, 1.000, 0.151), "windy": (0.048, None, None)}
    rows = []
    for a in X.columns:
        t = pd.crosstab(X[a], y).to_numpy()
        stat, p = id3.chi_square(t)
        rows.append({"attribute": a, "E(A) bits": id3.expected_information(t),
                     "gain bits": id3.information_gain(t), "paper gain": paper[a][0],
                     "IV bits": id3.intrinsic_value(t), "paper IV": paper[a][1],
                     "gain ratio": id3.gain_ratio(t), "paper gain ratio": paper[a][2],
                     "chi2": stat, "p-value": p})
    df = pd.DataFrame(rows).round(4)
    df.to_csv(os.path.join(RES, "e1_weather_gains.csv"), index=False)
    tree = id3.ID3().fit(X, y)
    tree_chi = id3.ID3(chi2_alpha=0.01).fit(X, y)
    with open(os.path.join(RES, "e1_weather_tree.txt"), "w") as f:
        f.write(f"I(9,5) = {id3.entropy([9, 5]):.4f} bits (paper 0.940)\n\n")
        f.write("ID3 tree (gain):\n" + tree.export_text() + "\n\n")
        f.write(f"Training accuracy {tree.score(X, y):.3f}; nodes {tree.n_nodes}, leaves {tree.n_leaves}\n\n")
        f.write("With the 99% chi-square test the tree is a single leaf:\n" + tree_chi.export_text() +
                "\n(14 objects are too few to reject independence)\n")
    return df, tree


# ---------------------------------------------------------------------------- E2 benchmark
MODELS = {
    "ID3 (gain)": dict(),
    "ID3 (gain ratio)": dict(criterion="gain_ratio"),
    "ID3 + chi2 99%": dict(chi2_alpha=0.01),
    "ID3 subsets (Sec. 7)": dict(split="subset"),
    "ID3 subsets + chi2 99%": dict(split="subset", chi2_alpha=0.01),
}


def _cart(name, Xtr, ytr, Xte, yte):
    from sklearn.tree import DecisionTreeClassifier
    clf = DecisionTreeClassifier(criterion="entropy", random_state=0)
    if is_numeric(name):
        clf.fit(Xtr, ytr)
        return clf.score(Xte, yte), clf.tree_.node_count
    from sklearn.preprocessing import OneHotEncoder
    enc = OneHotEncoder(handle_unknown="ignore")          # '?' becomes its own indicator
    clf.fit(enc.fit_transform(Xtr), ytr)
    return clf.score(enc.transform(Xte), yte), clf.tree_.node_count


def _e2_one(args):
    name, seed, train_frac = args
    X, y = load(name)
    Xtr, ytr, Xte, yte = split(X, y, 1 - train_frac, np.random.default_rng(seed))
    out = []
    for m, kw in MODELS.items():
        if is_numeric(name) and kw.get("split") == "subset":
            continue                                      # thresholds are already binary
        t0 = time.time(); tree = make(name, **kw).fit(Xtr, ytr)
        out.append(dict(dataset=name, seed=seed, train_frac=train_frac, model=m, accuracy=tree.score(Xte, yte),
                        nodes=tree.n_nodes, depth=tree.depth, fit_s=time.time() - t0, n_train=len(ytr), n_test=len(yte)))
    acc, nodes = _cart(name, Xtr, ytr, Xte, yte)
    out.append(dict(dataset=name, seed=seed, train_frac=train_frac, model="CART (sklearn)", accuracy=acc,
                    nodes=nodes, depth=np.nan, fit_s=np.nan, n_train=len(ytr), n_test=len(yte)))
    maj = pd.Series(ytr).value_counts().idxmax()
    out.append(dict(dataset=name, seed=seed, train_frac=train_frac, model="Majority class",
                    accuracy=float(np.mean(yte == maj)), nodes=1, depth=0, fit_s=0, n_train=len(ytr), n_test=len(yte)))
    return out


def e2_benchmark(pool, reps):
    """Seven real datasets, 70 % training, `reps` stratified splits; corrected paired t-tests."""
    jobs = [(n, s, 0.7) for n in BENCH for s in range(reps)]
    df = pd.DataFrame([r for res in pool.map(_e2_one, jobs) for r in res])
    df.to_csv(os.path.join(RES, "e2_benchmark_raw.csv"), index=False)
    summ = df.groupby(["dataset", "model"], sort=False).agg(
```

```
        acc_mean=("accuracy", "mean"), acc_se=("accuracy", se), nodes=("nodes", "mean")).reset_index()
    summ.round(4).to_csv(os.path.join(RES, "e2_benchmark_summary.csv"), index=False)

    tests = []
    for name in BENCH:
        d = df[df.dataset == name].pivot(index="seed", columns="model", values="accuracy")
        ntr, nte = df[df.dataset == name].n_train.iloc[0], df[df.dataset == name].n_test.iloc[0]
        pairs = [("ID3 (gain)", "CART (sklearn)"), ("ID3 + chi2 99%", "ID3 (gain)"), ("ID3 (gain ratio)", "ID3 (gain)")]
        if not is_numeric(name):
            pairs = [("ID3 subsets (Sec. 7)", "ID3 (gain)"), ("ID3 subsets (Sec. 7)", "CART (sklearn)")] + pairs
        for a, b in pairs:
            diff = d[a] - d[b]
            t, p = corrected_t(diff, ntr, nte)
            tests.append(dict(dataset=name, comparison=f"{a} - {b}", mean_diff_pts=100 * diff.mean(),
                              wins=int((diff > 0).sum()), ties=int((diff == 0).sum()), losses=int((diff < 0).sum()),
                              t=t, p_value=p))
    tests = pd.DataFrame(tests).round(4)
    tests.to_csv(os.path.join(RES, "e2_paired_tests.csv"), index=False)

    order = ["ID3 (gain)", "ID3 (gain ratio)", "ID3 + chi2 99%", "ID3 subsets (Sec. 7)", "CART (sklearn)"]
    col = {"ID3 (gain)": COLORS["id3"], "ID3 (gain ratio)": COLORS["ratio"], "ID3 + chi2 99%": COLORS["chi2"],
           "ID3 subsets (Sec. 7)": COLORS["subset"], "CART (sklearn)": COLORS["cart"]}
    fig, ax = plt.subplots(figsize=(10, 4.4))
    for i, name in enumerate(BENCH):
        for j, m in enumerate(order):
            r = summ[(summ.dataset == name) & (summ.model == m)]
            if len(r) == 0:
                continue
            ax.errorbar(100 * r.acc_mean, i + (j - 2) * 0.13, xerr=196 * r.acc_se, fmt="o", ms=5,
                        color=col[m], label=m if i == 0 else None)
        maj = summ[(summ.dataset == name) & (summ.model == "Majority class")].acc_mean.iloc[0]
        ax.plot(100 * maj, i, "|", color="#999", ms=14, label="majority class" if i == 0 else None)
    ax.set_yticks(range(len(BENCH))); ax.set_yticklabels([SHORT[n] for n in BENCH]); ax.invert_yaxis()
    ax.set_xlabel("test accuracy (%), mean and 95% CI over 20 splits (70% training)"); ax.set_xlim(55, 101)
    ax.legend(fontsize=8, loc="lower left", ncol=3)
    fig.tight_layout(); fig.savefig(os.path.join(FIG, "e2_benchmark.png")); plt.close(fig)
    return summ, tests


def _e2b_one(args):
    frac, seed = args
    X, y = load("chess")
    Xtr, ytr, Xte, yte = split(X, y, 1 - frac, np.random.default_rng(700 + seed))
    t = id3.ID3().fit(Xtr, ytr)
    return dict(train_frac=frac, seed=seed, accuracy=t.score(Xte, yte), nodes=t.n_nodes)


def e2b_learning_curve(pool, reps):
    """Section 4's experiment: train on a random fraction of the chess positions, test on the rest."""
    jobs = [(f, s) for f in (0.02, 0.05, 0.1, 0.2, 0.4, 0.7) for s in range(reps)]
    df = pd.DataFrame(list(pool.map(_e2b_one, jobs)))
    summ = df.groupby("train_frac").agg(accuracy=("accuracy", "mean"), acc_se=("accuracy", se),
                                        nodes=("nodes", "mean")).reset_index().round(4)
    summ.to_csv(os.path.join(RES, "e2b_chess_learning_curve.csv"), index=False)
    return summ


# ---------------------------------------------------------------------------- E3 depth + alpha
DEPTH_SETS = ["chess", "car", "votes", "bcl"]
ALPHAS = [None, 0.5, 0.2, 0.1, 0.05, 0.01, 0.001, 0.0001]


def _e3_one(args):
    name, seed = args
    X, y = load(name)
    Xtr, ytr, Xte, yte = split(X, y, 0.3, np.random.default_rng(2000 + seed))
    out = []
    for d in list(range(0, 9)) + [None]:
        t = make(name, max_depth=d).fit(Xtr, ytr)
        out.append(dict(dataset=name, seed=seed, control="depth", value=99 if d is None else d,
                        train_acc=t.score(Xtr, ytr), test_acc=t.score(Xte, yte), nodes=t.n_nodes))
    for a in ALPHAS:
        t = make(name, chi2_alpha=a).fit(Xtr, ytr)
        out.append(dict(dataset=name, seed=seed, control="alpha", value=1.0 if a is None else a,
                        train_acc=t.score(Xtr, ytr), test_acc=t.score(Xte, yte), nodes=t.n_nodes))
    return out


def e3_depth_alpha(pool, reps, sets=DEPTH_SETS, tag="real"):
    """Depth control and the chi-square confidence level."""
    jobs = [(n, s) for n in sets for s in range(reps)]
    df = pd.DataFrame([r for res in pool.map(_e3_one, jobs) for r in res])
    summ = df.groupby(["dataset", "control", "value"]).agg(
        train_acc=("train_acc", "mean"), test_acc=("test_acc", "mean"), test_se=("test_acc", se),
        nodes=("nodes", "mean")).reset_index().round(4)
    summ.to_csv(os.path.join(RES, f"e3_depth_alpha_{tag}.csv"), index=False)
    plot_e3(summ, sets, tag)
    return summ


def plot_e3(summ, sets, tag):
    fig, axes = plt.subplots(2, len(sets), figsize=(3.6 * max(len(sets), 2), 6.2), squeeze=False)
    for k, name in enumerate(sets):
```

```
        s = summ[(summ.dataset == name) & (summ.control == "depth")]
        ax = axes[0, k]; x = s.value.replace(99, 9)
        ax.plot(x, 100 * s.train_acc, "o-", ms=3, color="#999", label="train")
        ax.errorbar(x, 100 * s.test_acc, yerr=196 * s.test_se, fmt="o-", ms=3, color=COLORS["id3"], label="test")
        best = s.loc[s.test_acc.idxmax()]
        ax.plot(min(best.value, 9), 100 * best.test_acc, "o", ms=9, mfc="none", mec=COLORS["subset"], mew=1.5)
        ax.text(0.97, 0.45, f"best test: {'depth ' + str(int(best.value)) if best.value < 99 else 'full tree'}\n"
                f"({best.nodes:.0f} node{'' if round(best.nodes) == 1 else 's'})", transform=ax.transAxes, ha="right",
va="center", fontsize=8,
                color=COLORS["subset"], bbox=dict(boxstyle="round,pad=0.3", fc="white", ec="none", alpha=0.85))
        ax.set_xticks(range(10)); ax.set_xticklabels([str(i) for i in range(9)] + ["full"], fontsize=8)
        ax.set_title(SHORT.get(name, name)); ax.set_xlabel("max depth")
        s = summ[(summ.dataset == name) & (summ.control == "alpha")].sort_values("value", ascending=False)
        ax = axes[1, k]; xs = np.arange(len(s))
        ax2 = ax.twinx(); ax2.bar(xs, s.nodes, color="#cccccc", alpha=0.6, width=0.5)
        ax2.set_ylabel("nodes (bars)", fontsize=8, color="#888"); ax2.tick_params(labelsize=7, colors="#888"); ax2.grid(False)
        ax.set_zorder(ax2.get_zorder() + 1); ax.patch.set_visible(False)
        ax.errorbar(xs, 100 * s.test_acc, yerr=196 * s.test_se, fmt="o-", ms=3, color=COLORS["chi2"])
        ax.set_xticks(xs); ax.set_xticklabels(["none" if v == 1 else f"{v:g}" for v in s.value], rotation=45, fontsize=7)
        ax.set_xlabel("χ² level α (split only if p < α)")
    axes[0, 0].set_ylabel("accuracy (%)"); axes[1, 0].set_ylabel("test accuracy (%)"); axes[0, 0].legend(fontsize=8)
    for k in range(len(sets), axes.shape[1]):
        axes[0, k].axis("off"); axes[1, k].axis("off")
    fig.tight_layout(); fig.savefig(os.path.join(FIG, f"e3_depth_alpha_{tag}.png")); plt.close(fig)


# ---------------------------------------------------------------------------- E4 noise
NOISE_LEVELS = [0.0, 0.05, 0.1, 0.15, 0.2, 0.3, 0.4, 0.5, 0.6, 0.8, 1.0]
PAPER_T2 = pd.DataFrame({"level": [5, 10, 15, 20, 30, 40, 50, 60, 70, 80, 90, 100],
                         "single": [1.3, 2.5, 3.3, 4.6, 6.1, 7.6, 8.8, 9.4, 9.9, 10.4, 10.8, 10.8],
                         "all": [11.9, 18.9, 24.6, 27.8, 29.5, 30.3, 29.2, 27.5, 25.9, 26.0, 25.6, 25.9],
                         "class": [2.6, 5.5, 8.3, 9.9, 14.8, 18.1, 21.8, 26.4, 27.2, 29.5, 34.1, 49.6]})


def root_attribute(name):
    X, y = load(name)
    return id3.ID3(max_depth=1).fit(X, y).root_.attribute


def _e4_one(args):
    name, where, level, alpha, seed, single_attr = args
    X, y = load(name)
    rng = np.random.default_rng(1000 + seed)
    Xtr, ytr, Xte, yte = split(X, y, 0.3, rng)
    dom = {a: sorted(X[a].unique()) for a in X.columns}
    cls = np.unique(y)
    Xtr_c, ytr_c = id3.corrupt(Xtr, ytr, level, where, dom, cls, rng, attribute=single_attr)
    Xte_c, yte_c = id3.corrupt(Xte, yte, level, where, dom, cls, rng, attribute=single_attr)
    tree = id3.ID3(chi2_alpha=alpha).fit(Xtr_c, ytr_c)
    row = dict(dataset=name, where=where, level=level, chi2=alpha is not None, seed=seed,
               error=1 - tree.score(Xte_c, yte_c), nodes=tree.n_nodes, error_clean_tree=np.nan)
    if where == "all" and alpha is not None:
        clean = id3.ID3(chi2_alpha=alpha).fit(Xtr, ytr)          # the paper's "laboratory" tree
        row["error_clean_tree"] = 1 - clean.score(Xte_c, yte_c)
    return row


def e4_noise(pool, reps, sets=("chess", "car"), tag="real"):
    """Replicates Table 2: noise in the root attribute, in all attributes, and in the class."""
    single = {n: root_attribute(n) for n in sets}
    jobs = [(n, w, l, a, s, single[n]) for n in sets for w in ("single", "all", "class")
            for l in NOISE_LEVELS for a in (0.01, None) for s in range(reps)]
    df = pd.DataFrame(list(pool.map(_e4_one, jobs, chunksize=4)))
    summ = df.groupby(["dataset", "where", "chi2", "level"]).agg(
        error=("error", "mean"), error_se=("error", se), nodes=("nodes", "mean"),
        error_clean_tree=("error_clean_tree", "mean")).reset_index().round(4)
    summ["single_attribute"] = summ.dataset.map(single)
    summ.to_csv(os.path.join(RES, f"e4_noise_{tag}.csv"), index=False)
    summ[summ.chi2].pivot_table(index="level", columns=["dataset", "where"], values="error") \
        .to_csv(os.path.join(RES, f"e4_table2_{tag}.csv"))
    PAPER_T2.to_csv(os.path.join(RES, "paper_table2.csv"), index=False)

    fig, axes = plt.subplots(1, len(sets) + 1, figsize=(4.7 * (len(sets) + 1), 3.8))
    for ax, name in zip(axes, sets):
        for w in ("single", "all", "class"):
            s = summ[(summ.dataset == name) & (summ["where"] == w) & summ.chi2]
            ax.errorbar(s.level * 100, s.error * 100, yerr=196 * s.error_se, fmt="o-", ms=3, color=COLORS[w],
                        label={"single": f"one attribute ({single[name]})", "all": "all attributes", "class": "class"}[w])
        ax.set_title(f"{SHORT[name]}: ID3 + χ² 99%"); ax.set_xlabel("noise level (%)"); ax.set_ylabel("test error (%)")
        ax.legend(fontsize=8)
    for w in ("single", "all", "class"):
        axes[-1].plot(PAPER_T2.level, PAPER_T2[w], "o-", ms=3, color=COLORS[w], label=w)
    axes[-1].set_title("Quinlan (1986) Table 2 (chess, 551 objects)"); axes[-1].set_xlabel("noise level (%)")
    axes[-1].legend(fontsize=8)
    fig.tight_layout(); fig.savefig(os.path.join(FIG, f"e4_noise_{tag}.png")); plt.close(fig)

    name = sets[0]
    fig, axes = plt.subplots(1, 2, figsize=(10, 3.6))
    for ax, w in zip(axes, ("all", "class")):
        for chi, c, lab in ((True, COLORS["chi2"], "with χ² test (99%)"), (False, COLORS["id3"], "no test")):
            s = summ[(summ.dataset == name) & (summ["where"] == w) & (summ.chi2 == chi)]
            ax.plot(s.level * 100, s.error * 100, "o-", ms=3, color=c, label=lab)
        if w == "all":
```

```
            s = summ[(summ.dataset == name) & (summ["where"] == "all") & summ.chi2]
            ax.plot(s.level * 100, s.error_clean_tree * 100, "s--", ms=3, color="#555", label="tree built on clean data")
        ax.set_title(f"{SHORT[name]}: noise in {'all attributes' if w == 'all' else 'the class'}")
        ax.set_xlabel("noise level (%)"); ax.set_ylabel("test error (%)"); ax.legend(fontsize=8)
    fig.tight_layout(); fig.savefig(os.path.join(FIG, f"e4_chi2_ablation_{tag}.png")); plt.close(fig)
    return summ


# ------------------------------------------------------------------------------ E5 unknowns
IGNORANCE = [0.0, 0.05, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6]


def mode_impute(Xtr, Xte):
    """The paper's 'simple strategy': replace an unknown by the attribute's most common value."""
    Xtr, Xte = Xtr.copy().astype(object), Xte.copy().astype(object)
    for a in Xtr.columns:
        known = Xtr[a][Xtr[a] != "?"]
        m = known.mode().iloc[0] if len(known) else "?"
        Xtr[a] = Xtr[a].replace("?", m); Xte[a] = Xte[a].replace("?", m)
    return Xtr, Xte


def as_value(D):
    v = D.to_numpy(dtype=object)
    return pd.DataFrame(np.where(v == "?", "UNK", v), columns=D.columns, dtype=object)


def _three_ways(Xtr, ytr, Xte, yte):
    out = {"Quinlan Sec. 6": id3.ID3().fit(Xtr, ytr).score(Xte, yte),
           "unknown as a value": id3.ID3().fit(as_value(Xtr), ytr).score(as_value(Xte), yte)}
    a, b = mode_impute(Xtr, Xte)
    out["most common value"] = id3.ID3().fit(a, ytr).score(b, yte)
    return out


def _e5_one(args):
    name, level, seed = args
    X, y = load(name)
    rng = np.random.default_rng(5000 + seed)
    Xtr, ytr, Xte, yte = split(X, y, 0.3, rng)
    Xtr_u, Xte_u = id3.blank_out(Xtr, level, rng), id3.blank_out(Xte, level, rng)
    out = dict(dataset=name, level=level, seed=seed)
    for k, v in _three_ways(Xtr_u, ytr, Xte_u, yte).items():
        out["error: " + k] = 1 - v
    out["error: clean tree, incomplete objects"] = 1 - id3.ID3().fit(Xtr, ytr).score(Xte_u, yte)
    return out


def _e5_real(args):
    name, seed = args
    X, y = load(name)
    Xtr, ytr, Xte, yte = split(X, y, 0.3, np.random.default_rng(9000 + seed))
    return dict(dataset=name, seed=seed, **_three_ways(Xtr, ytr, Xte, yte))


def e5_unknown(pool, reps, sets=("chess", "car"), tag="real"):
    """Figure 5 (ignorance level) on real data, and three strategies on genuinely missing data."""
    jobs = [(n, l, s) for n in sets for l in IGNORANCE for s in range(reps)]
    df = pd.DataFrame(list(pool.map(_e5_one, jobs, chunksize=4)))
    summ = df.groupby(["dataset", "level"]).mean(numeric_only=True).drop(columns="seed").reset_index().round(4)
    summ.to_csv(os.path.join(RES, f"e5_ignorance_{tag}.csv"), index=False)

    real = pd.DataFrame(list(pool.map(_e5_real, [(n, s) for n in ("votes", "mushroom", "bcl") for s in range(reps)])))
    rs = real.drop(columns="seed").groupby("dataset", sort=False).agg(["mean", se])
    rs.columns = [f"{a} ({'mean' if b == 'mean' else 'se'})" for a, b in rs.columns]
    rs = rs.reset_index().round(4)
    rs.to_csv(os.path.join(RES, "e5_real_missing.csv"), index=False)

    fig, axes = plt.subplots(1, len(sets) + 1, figsize=(4.6 * (len(sets) + 1), 3.6))
    styles = {"error: Quinlan Sec. 6": ("o-", COLORS["id3"]), "error: unknown as a value": ("o-", COLORS["all"]),
              "error: most common value": ("^-", COLORS["chi2"]), "error: clean tree, incomplete objects": ("s--", "#555")}
    for ax, name in zip(axes, sets):
        s = summ[summ.dataset == name]
        for c, (fmt, colr) in styles.items():
            ax.plot(s.level * 100, s[c] * 100, fmt, ms=3, color=colr, label=c.replace("error: ", ""))
        ax.set_title(f"{SHORT[name]}: unknown values"); ax.set_xlabel("ignorance level (%)"); ax.set_ylabel("test error (%)")
    axes[0].legend(fontsize=7)
    ax = axes[-1]
    names = list(rs.dataset); xs = np.arange(len(names))
    for j, (k, colr) in enumerate([("Quinlan Sec. 6", COLORS["id3"]), ("unknown as a value", COLORS["all"]),
                                   ("most common value", COLORS["chi2"])]):
        ax.bar(xs + (j - 1) * 0.26, 100 * (1 - rs[f"{k} (mean)"]), width=0.26, yerr=196 * rs[f"{k} (se)"],
               color=colr, label=k)
    ax.set_xticks(xs); ax.set_xticklabels([SHORT[n] for n in names], fontsize=8)
    ax.set_ylabel("test error (%)"); ax.set_title("Genuinely missing values (real data)"); ax.legend(fontsize=7)
    fig.tight_layout(); fig.savefig(os.path.join(FIG, f"e5_unknown_{tag}.png")); plt.close(fig)
    return summ, rs


# ------------------------------------------------------------------------------ E6 gain bias
def e6_gain_bias(reps):
    """Section 7: a many-valued irrelevant attribute fools the gain criterion, not the gain ratio
    (simulated sanity check, because the truth is known)."""
```

```
    X, y = load("sim_id")
    rows = []
    for s in range(reps):
        Xtr, ytr, Xte, yte = split(X, y, 0.3, np.random.default_rng(300 + s))
        for crit, kw in (("gain", {}), ("gain ratio", {"criterion": "gain_ratio"}), ("gain, binary subsets", {"split":
"subset"})):
            t = id3.ID3(**kw).fit(Xtr, ytr)
            uses = sum(1 for n in t._walk() if n.attribute == "id_code")
            rows.append(dict(seed=s, criterion=crit, root=t.root_.attribute, id_code_tests=uses,
                             accuracy=t.score(Xte, yte), nodes=t.n_nodes))
    df = pd.DataFrame(rows)
    summ = df.groupby("criterion", sort=False).agg(
        id_code_tests=("id_code_tests", "mean"), accuracy=("accuracy", "mean"), acc_se=("accuracy", se),
        nodes=("nodes", "mean")).reset_index().round(4)
    summ.to_csv(os.path.join(RES, "e6_gain_bias.csv"), index=False)
    Xtr, ytr, _, _ = split(X, y, 0.3, np.random.default_rng(300))
    g = []
    for a in X.columns:
        t = pd.crosstab(Xtr[a], ytr).to_numpy()
        g.append(dict(attribute=a, values=Xtr[a].nunique(), gain=id3.information_gain(t),
                      IV=id3.intrinsic_value(t), gain_ratio=id3.gain_ratio(t)))
    pd.DataFrame(g).sort_values("gain", ascending=False).round(4).to_csv(os.path.join(RES, "e6_root_scores.csv"), index=False)
    return summ


# ---------------------------------------------------------------------------- E7 windowing
def e7_windowing():
    """Section 4: the window, on the chess endgame and mushroom data."""
    rows, hists = [], {}
    for name in ("chess", "mushroom"):
        X, y = load(name)
        Xtr, ytr, Xte, yte = split(X, y, 0.3, np.random.default_rng(11))
        t0 = time.time(); direct = id3.ID3().fit(Xtr, ytr); td = time.time() - t0
        t0 = time.time(); win, hist = id3.fit_windowed(lambda: id3.ID3(), Xtr, ytr, window=100, add=50, seed=0); tw = time.time()
- t0
        hists[name] = hist
        rows += [dict(dataset=name, method="direct", objects_used=len(ytr), nodes=direct.n_nodes,
                      train_errors=int((direct.predict(Xtr) != ytr).sum()), test_accuracy=direct.score(Xte, yte), seconds=td),
                 dict(dataset=name, method="windowing (100 + 50 per round)", objects_used=hist[-1]["window"],
                      nodes=win.n_nodes, train_errors=hist[-1]["errors"], test_accuracy=win.score(Xte, yte), seconds=tw)]
    out = pd.DataFrame(rows).round(4)
    out.to_csv(os.path.join(RES, "e7_windowing.csv"), index=False)
    pd.concat([pd.DataFrame(h).assign(dataset=n) for n, h in hists.items()]).to_csv(
        os.path.join(RES, "e7_windowing_history.csv"), index=False)
    return out


# ---------------------------------------------------------------------------- E8 theory
def e8_theory():
    """Section 5's explanation of the error peak: probabilistic leaf vs majority leaf."""
    q = np.linspace(0.5, 1.0, 51)
    fig, ax = plt.subplots(figsize=(4.8, 3.4))
    ax.plot(q, 2 * q * (1 - q), color=COLORS["all"], label="classify P with prob. p/(p+n):  2pn/(p+n)²")
    ax.plot(q, 1 - q, color=COLORS["id3"], label="always the majority class:  n/(p+n)")
    ax.set_xlabel("p/(p+n)"); ax.set_ylabel("expected error"); ax.legend(fontsize=7)
    ax.set_title("Why error falls again at high noise")
    fig.tight_layout(); fig.savefig(os.path.join(FIG, "e8_theory_peak.png")); plt.close(fig)


# ---------------------------------------------------------------------------- sanity checks
def sanity_checks(pool, reps):
    """Simulated data, where the truth is known: recovery of the generating tree, depth and noise."""
    from simulate import TRUE_TREE_TEXT
    X, y = load("sim")
    t = id3.ID3().fit(X, y)
    recovered = "\n".join(l.split("  (n=")[0] for l in t.export_text().split("\n"))
    ok = recovered.strip() == TRUE_TREE_TEXT.strip()
    with open(os.path.join(RES, "s1_sim_recovery.txt"), "w") as f:
        f.write(f"ID3 on 2000 simulated objects recovers the generating tree exactly: {ok}\n\n{t.export_text()}\n")
    e3_depth_alpha(pool, reps, sets=["sim"], tag="sim")
    e4_noise(pool, reps, sets=("sim",), tag="sim")
    return ok


# ---------------------------------------------------------------------------- tree figure
def draw_weather_tree(tree, path=os.path.join(FIG, "weather_tree.png")):
    """Hand-drawn layout of the Figure-2 tree learned from Table 1."""
    fig, ax = plt.subplots(figsize=(7, 3.6)); ax.axis("off")
    box = dict(boxstyle="round,pad=0.4", fc="#e8f0f8", ec="#2a7ab9")
    leafP = dict(boxstyle="round,pad=0.3", fc="#e6f4ea", ec="#3c9a5f")
    leafN = dict(boxstyle="round,pad=0.3", fc="#fbe9eb", ec="#d1495b")

    def text(x, yv, s, b):
        ax.text(x, yv, s, ha="center", va="center", bbox=b, fontsize=10)

    def edge(x0, y0, x1, y1, lab):
        ax.plot([x0, x1], [y0 - 0.06, y1 + 0.06], color="#777", lw=1)
        ax.text((x0 + x1) / 2, (y0 + y1) / 2, lab, fontsize=8, ha="center", va="center",
                color="#333", bbox=dict(fc="white", ec="none", pad=0.5))

    root = tree.root_
    text(0.5, 0.9, f"outlook\ngain {root.gain:.3f}", box)
    for v, x in {"sunny": 0.17, "overcast": 0.5, "rain": 0.83}.items():
```

```
        child = root.children[v]
        edge(0.5, 0.9, x, 0.55, v)
        if child.is_leaf:
            lab = tree.classes_[child.dist.argmax()]
            text(x, 0.55, f"{lab}  ({int(child.dist.sum())})", leafP if lab == "P" else leafN)
        else:
            text(x, 0.55, f"{child.attribute}\ngain {child.gain:.3f}", box)
            for k, (val, g) in enumerate(child.children.items()):
                xx = x + (-0.11 if k == 0 else 0.11)
                edge(x, 0.55, xx, 0.15, val)
                lab = tree.classes_[g.dist.argmax()]
                text(xx, 0.15, f"{lab}  ({int(g.dist.sum())})", leafP if lab == "P" else leafN)
    ax.set_xlim(0, 1); ax.set_ylim(0, 1)
    fig.tight_layout(); fig.savefig(path); plt.close(fig)


# ---------------------------------------------------------------------------- data summary
def data_summary():
    rows = []
    for name, (f, ycol, label, kind, numeric) in DATASETS.items():
        X, y = load(name)
        vc = pd.Series(y).value_counts()
        n_unknown = int(sum((X[a].astype(str) == "?").sum() for a in X.columns))
        nvals = "continuous" if numeric else f"{min(X[a].nunique() for a in X.columns)}-{max(X[a].nunique() for a in X.columns)}"
        rows.append({"dataset": label, "file": f, "type": kind, "objects": len(y), "attributes": X.shape[1],
                     "values per attribute": nvals, "classes": ", ".join(f"{k} {v}" for k, v in vc.items()),
                     "unknown values": n_unknown})
    df = pd.DataFrame(rows)
    df.to_csv(os.path.join(RES, "data_summary.csv"), index=False)
    return df


def main(quick=False):
    reps = 5 if quick else REPS
    t0 = time.time()
    lap = lambda s: print(f"[{time.time() - t0:5.0f}s] {s}", flush=True)
    print(data_summary().to_string(index=False))
    df1, tree = e1_weather(); draw_weather_tree(tree); print(df1.to_string(index=False))
    e8_theory()
    with ProcessPoolExecutor(max_workers=os.cpu_count()) as pool:
        summ, tests = e2_benchmark(pool, reps); lap("E2 benchmark")
        print(summ.pivot(index="model", columns="dataset", values="acc_mean").round(4).to_string())
        print(tests.to_string(index=False))
        print(e2b_learning_curve(pool, reps).to_string(index=False)); lap("E2b learning curve")
        e3_depth_alpha(pool, reps); lap("E3 depth/alpha")
        e4_noise(pool, reps); lap("E4 noise")
        e5_unknown(pool, reps); lap("E5 unknown")
        print("sanity: simulated tree recovered =", sanity_checks(pool, reps)); lap("sanity checks")
    print(e6_gain_bias(reps).to_string(index=False))
    print(e7_windowing().to_string(index=False)); lap("all done")


if __name__ == "__main__":
    ap = argparse.ArgumentParser()
    ap.add_argument("--quick", action="store_true")
    main(ap.parse_args().quick)
```

## tests/test_id3.py

```
"""Unit tests: each one checks a specific claim or number from Quinlan (1986).

    python -m pytest -q tests
"""
import os
import sys

import numpy as np
import pandas as pd
import pytest
from scipy.stats import chi2_contingency

HERE = os.path.dirname(os.path.abspath(__file__))
ROOT = os.path.dirname(HERE)
sys.path.insert(0, ROOT)
import id3  # noqa: E402
from simulate import TRUE_TREE_TEXT, simulate  # noqa: E402


@pytest.fixture(scope="module")
def weather():
    d = pd.read_csv(os.path.join(ROOT, "data/real/weather_quinlan1986.csv"), dtype=str)
    return d[["outlook", "temperature", "humidity", "windy"]], d["class"]


def test_section4_information_of_table1():
    assert id3.entropy([9, 5]) == pytest.approx(0.940, abs=5e-4)          # paper: 0.940 bits


def test_section4_gains_of_table1(weather):
    X, y = weather
    paper = {"outlook": 0.246, "temperature": 0.029, "humidity": 0.151, "windy": 0.048}
    for a, g in paper.items():
        # the paper rounds intermediate results (0.940 - 0.694), so allow 1e-3
```

```
        assert id3.information_gain(pd.crosstab(X[a], y).to_numpy()) == pytest.approx(g, abs=1.1e-3)


def test_section4_tree_is_figure2(weather):
    X, y = weather
    t = id3.ID3().fit(X, y)
    assert t.root_.attribute == "outlook"
    assert t.root_.children["sunny"].attribute == "humidity"
    assert t.root_.children["rain"].attribute == "windy"
    assert t.root_.children["overcast"].is_leaf
    assert t.score(X, y) == 1.0 and t.n_leaves == 5


def test_section7_gain_ratio_numbers(weather):
    X, y = weather
    t = pd.crosstab(X["outlook"], y).to_numpy()
    assert id3.intrinsic_value(t) == pytest.approx(1.578, abs=1e-3)
    assert id3.gain_ratio(t) == pytest.approx(0.156, abs=1e-3)
    h = pd.crosstab(X["humidity"], y).to_numpy()
    assert id3.gain_ratio(h) == pytest.approx(0.151, abs=1.1e-3)


def test_footnote3_gain_is_mutual_information():
    rng = np.random.default_rng(0)
    t = rng.integers(0, 20, size=(4, 3)).astype(float)
    p = t / t.sum()
    px, py = p.sum(1, keepdims=True), p.sum(0, keepdims=True)
    with np.errstate(divide="ignore", invalid="ignore"):
        mi = np.nansum(p * np.log2(p / (px * py)))
    assert id3.information_gain(t) == pytest.approx(mi, abs=1e-12)


def test_section5_chi_square_matches_scipy():
    rng = np.random.default_rng(1)
    for _ in range(20):
        t = rng.integers(1, 30, size=(rng.integers(2, 5), rng.integers(2, 4))).astype(float)
        stat, p = id3.chi_square(t)
        s2, p2, _, _ = chi2_contingency(t, correction=False)
        assert stat == pytest.approx(s2) and p == pytest.approx(p2)


def test_section7_splitting_a_value_never_lowers_gain():
    """Paper: gain(A') >= gain(A) when A' splits one value of A into two."""
    rng = np.random.default_rng(2)
    for _ in range(500):
        t = rng.integers(0, 15, size=(3, 2)).astype(float)
        cut = rng.integers(0, t[0] + 1)
        t2 = np.vstack([cut, t[0] - cut, t[1:]])
        assert id3.information_gain(t2) >= id3.information_gain(t) - 1e-12


def test_section6_unknowns_can_only_lower_gain():
    """Paper: the proportional treatment of unknown values can only decrease the gain."""
    rng = np.random.default_rng(3)
    for _ in range(500):
        t = rng.integers(1, 20, size=(3, 2)).astype(float)
        u = rng.integers(0, 10, size=2).astype(float)
        assert id3.information_gain(id3._adjust(t, u)) <= id3.information_gain(t) + 1e-12


def test_section6_unknown_as_value_anomaly():
    """Paper's example: E(A) = 1 bit, but E(A') = 0.84 bits once one value becomes 'unknown'."""
    assert id3.expected_information([[2, 2], [2, 2]]) == pytest.approx(1.0)
    assert id3.expected_information([[1, 2], [2, 2], [1, 0]]) == pytest.approx(0.84, abs=0.01)


def test_section6_token_splitting_weights(weather):
    X, y = weather
    t = id3.ID3().fit(X, y)
    row = pd.DataFrame([{"outlook": "?", "temperature": "hot", "humidity": "high", "windy": "false"}])
    # sunny (5/14) -> N, overcast (4/14) -> P, rain (5/14) -> P (windy = false)
    p = t.predict_proba(row)[0]
    assert p[list(t.classes_).index("P")] == pytest.approx(9 / 14)


def test_simulated_tree_is_recovered():
    df = simulate(2000, seed=2026)
    t = id3.ID3().fit(df.drop(columns="class"), df["class"])
    text = "\n".join(l.split("  (n=")[0] for l in t.export_text().split("\n"))
    assert text.strip() == TRUE_TREE_TEXT.strip()


def test_chi2_test_blocks_irrelevant_attributes():
    rng = np.random.default_rng(4)
    X = pd.DataFrame({f"R{i}": rng.choice(list("abc"), 500) for i in range(5)})
    y = rng.choice(["P", "N"], 500)
    assert id3.ID3(chi2_alpha=0.01).fit(X, y).n_nodes == 1     # pure noise -> single leaf
    assert id3.ID3().fit(X, y).n_nodes > 50                    # without the test it fits the noise


def test_threshold_at_midpoint():
    X = pd.DataFrame({"x": [1.0, 2.0, 3.0, 10.0, 11.0, 12.0]})
    y = ["N", "N", "N", "P", "P", "P"]
```

```
    t = id3.ID3(continuous=["x"]).fit(X, y)
    assert t.root_.kind == "threshold" and t.root_.param == pytest.approx(6.5)
    assert list(t.predict(pd.DataFrame({"x": [0.0, 6.0, 7.0, 99.0]}))) == ["N", "N", "P", "P"]


def test_subset_split_is_binary_and_exact():
    X = pd.DataFrame({"c": list("abcdab")})
    y = ["P", "N", "P", "N", "P", "N"]          # a, c -> P ; b, d -> N
    t = id3.ID3(split="subset").fit(X, y)
    assert t.root_.kind == "subset" and set(t.root_.children) == {"in", "out"}
    assert t.score(X, y) == 1.0 and t.n_nodes == 3


def test_fit_does_not_modify_input(weather):
    X, y = weather
    Xu = X.copy(); Xu.iloc[0, 0] = "?"
    before = Xu.copy()
    id3.ID3().fit(Xu, y)
    pd.testing.assert_frame_equal(Xu, before)
```